



AgentOS-uCore 设计开发文档

基于 RISC-V uCore 的系统原生智能体运行机制

项目名称	AgentOS-uCore
队伍名称	happy-legend
目标平台	RISC-V 64 / uCore / QEMU
文档版本	决赛设计开发文档

姓名	联系方式 (QQ)
王浩泮	2091576055
康俊豪	488718235

2026 年 08 月

摘要

Agent 工作流已从一次性对话扩展为持续运行的工具调用、文件处理与多角色协作。现有应用层框架能够编排模型和工具，却无法在系统效果发生的位置统一表达角色、工作流世代、上下文因果、资源归属和 CPU 服务。仅依赖 prompt、JSON 与应用日志时，进程、文件、IPC、调度之间的关系会分散在不同层次，难以形成可执行的内核约束。

本项目基于 LearningOS/uCore 实现 AgentOS-uCore。系统以 RISC-V uCore 的进程、虚拟内存、VFS、系统调用和调度器为基础，将 Agent 定义为内核可识别的工作负载：身份与 lifecycle 将受信映像、role、capability、workflow id+generation 绑定到进程；结构化工具执行将目录、参数 schema、执行合同和 Task SQ/CQ 置于统一 UAPI；Context Path、provenance 与 Evidence Ring 保存请求、结果及数据来源；Live-Query 文件系统为显式登记的对象提供查询与订阅；Agent Loop 与 Workflow EEVDF 将等待、消息、资源账户和 CPU 服务组织为工作流级实体。

在产品集成层，AgentOS Console 组织确定性科研恢复流程，AgentOS Nexus 在同一 QEMU boot 中维持 Coordinator、System、Research、Analyst 四个业务角色。Nexus 通过 relay、结构化工具、消息等待和 artifact 数据面完成动态委派；Guest 侧可以同时观察 identity、Context、文件、IPC、等待和调度状态。

一次性增强实验使用 30 个 fresh QEMU boot，保存 33 份原始串口输出、19 个 CSV 数据表和 7,498 条逐样本记录。16 个独立 workflow 配对中，indexed core path 均低于 traversal，配对加速比中位数为 **3.118 倍**。文件查询网格覆盖 12 组 catalog size 与 hit count 组合，每组保留 15 个 AB/BA inner pair；Task 实验保留 96 条 16-op sequence；EEVDF 实验保留 504 条精确唤醒探针，均在 0–1 tick 内得到服务。

赛题目标	完成情况	实现说明
使操作系统识别 Agent 运行主体	全部完成	受信映像创建、role/capability、control id、workflow lifecycle 与资源账户均进入内核对象。
形成受控工具调用路径	全部完成	工具目录、V2/V3 请求、schema 解码、执行合同、重试与 Task Channel 使用定长 UAPI 对接。
维护上下文与数据来源	全部完成	Context mirror、路径索引、branch/rollback、provenance、Evidence Ring 和 Workflow Fence 形成连续记录。
提供文件查询与实时事件	全部完成	metadata catalog、选择性索引、typed query/watch、增量缺口 resync 与 VFS 事件更新已实现。
支持等待、协作与资源调度	全部完成	MESSAGE route、atomic wait/wake、heartbeat、Credit Domain 和 Workflow EEVDF 已接入 Guest 场景。
验证端到端运行与性能	全部完成	Console 恢复工作流、Nexus 四角色协作、Guest 合同检查与一次性增强实验共同验证。

表 1: AgentOS-uCore 赛题目标及完成情况

Contents

摘要	ii
第一章 项目背景	1
1.1 项目背景与设计目标	1
1.1.1 Agent 对操作系统提出的新需求	1
1.1.2 基于 uCore 的系统扩展	1
1.1.3 机制与策略的分层	2
1.1.4 相关机制与技术选择	2
1.2 设计目标与验收线索	2
第二章 系统设计	4
2.1 AgentOS-uCore 总体架构	4
2.1.1 以 uCore 为底座的产品分层	4
2.1.2 六项核心机制	5
2.1.3 一次 Agent 工具调用	5
2.1.4 上层运行时	5
2.2 跨模块状态约束	6
第三章 系统实现	7
3.1 内核原生 Agent 身份与生命周期	7
3.1.1 从进程身份到 workflow 主体	7
3.1.2 身份字段与角色策略	8
3.1.3 七页 Context 地址区	8
3.1.4 Lifecycle generation 与并发 gate	8
3.1.5 资源主体随 workflow 创建	9
3.1.6 验证结果	9
3.2 结构化工具执行	9
3.2.1 工具目录与参数模式	9
3.2.2 Exploratory typed V2	9
3.2.3 四种执行路径	9
3.2.4 Tool Phase Credit Lease	10
3.2.5 Typed Task Channel	10
3.2.6 验证结果	10
3.3 Context Path、Provenance 与 Workflow Fence	11
3.3.1 从多轮文本到结构化 Context	11
3.3.2 分支、回退与淘汰	11
3.3.3 数据来源传播	11
3.3.4 工具 Manifest 与效果检查	11
3.3.5 Evidence Ring	12
3.3.6 Workflow Fence	12
3.3.7 验证结果	12
3.4 Agent Live-Query 文件系统	13
3.4.1 Agent 对文件对象的结构化需求	13
3.4.2 Metadata 与 inode incarnation	13
3.4.3 Traversal 与选择性索引	13
3.4.4 Typed live watch 与增量缺口恢复	13
3.4.5 查询性能	14
3.4.6 验证结果	15
3.5 Agent Loop 与 Workflow EEVDF	15
3.5.1 从轮询循环到内核等待	15
3.5.2 Heartbeat 与 LLM correlation	15
3.5.3 Workflow Credit Domain	15
3.5.4 按 workflow 聚合的 EEVDF	16
3.5.5 唤醒延迟与公平性	16
3.5.6 验证结果	16
3.6 AgentOS Console 综合工作流	17
3.6.1 科研恢复场景	17
3.6.2 Traversal 与 Indexed 配对	17
3.6.3 Core 与端到端窗口	17
3.6.4 系统结果	18
3.7 AgentOS Nexus 多智能体协作平台	18
3.7.1 长驻会话与四角色协作	18
3.7.2 一次交互回合	19
3.7.3 N1 子任务协议	20
3.7.4 Artifact 数据面	20
3.7.5 动态工具呈现	20

3.7.6	调用级审批与取消	20
3.7.7	内核观测面	21
3.7.8	运行验证	21
3.8	关键实现证据	21
3.8.1	启动、身份与 workflow 世代	21
3.8.2	结构化工具调用与执行合同	25
3.8.3	Context、回滚边界与 provenance	32
3.8.4	Metadata 查询与 Live Query 事件	36
3.8.5	Task Channel 与等待路径	43
3.8.6	Phase Lease 与 Workflow EEVDF	45
3.8.7	Guest 运行时与工件发布	52
第四章	项目测试	57
4.1	功能与性能测试	57
4.1.1	测试组织	57
4.1.2	一次性增强实验	57
4.1.3	Task 传输延迟	58
4.1.4	内核 I/O 与工作量	58
4.1.5	可重复性	59
4.2	测试路径与可复核证据	59
4.2.1	Native workload 与一次性测量	59
4.2.2	工具、合同与任务通道测试	61
4.2.3	Nexus 会话与 Host relay	65
4.2.4	测试结论的解释范围	67
第五章	总结与展望	68
5.1	阶段性结论	68
5.2	当前技术限制	68
5.3	后续工作	68
第六章	参考说明	69
6.1	uCore 与系统基础	69
6.2	协议、工具与运行时	69
6.3	测量资料	69
6.4	文献使用	69
第七章	参考文献	70

1. 项目背景

智能体的“计划-调用-等待-修正”循环同时穿过进程、文件、系统调用和调度器。若角色、 workflow 标识和来源关系仅存于应用层，内核只能看到无差别的线程和文件描述符，无法在创建、效果提交、资源占用或唤醒服务时执行同一套判断。本章说明 AgentOS-uCore 的问题来源、uCore 扩展边界和目标分解。

1.1. 项目背景与设计目标

1.1.1. Agent 对操作系统提出的新需求

大模型工具调用已经从单次问答发展为持续运行的 Agent Loop。一个典型工作流会反复执行“读取上下文、选择工具、等待结果、修正计划”，并将任务拆分给多个专职 Agent。这种负载具有三个突出特点：生命周期长，工具副作用丰富，协作中间状态多。

传统进程抽象保存 PID、内存、文件和 CPU 时间，却无法直接表达 Agent 的角色、工作流世代、上下文因果、工具模式和数据来源。这些信息若只保存在 prompt、JSON 和应用日志中，文件、IPC 与资源操作就难以在实际发生前得到统一检查。一个角色还可以通过创建更多线程放大调度份额，进而影响其他工作流的服务时间。

我们将 AgentOS-uCore 的目标归纳为一句话：**让操作系统识别 Agent 工作负载，并在系统效果发生处提供一致的身份、资源、因果和调度机制。**

1.1.2. 基于 uCore 的系统扩展

项目建立在 LearningOS/uCore 教学内核之上 [1]。uCore 提供 RISC-V 启动、进程、虚拟内存、VFS、trap、系统调用和基础调度机制。我们沿用这些主干，在现有 PCB、VFS 对象、调度队列和 syscall 路径上增加 AgentOS 状态。所有新 UAPI 使用定长结构、显式 version/size 和 kernel/user 共用布局检查。普通 uCore 进程继续沿用原有路径，Agent 进程则在受控的创建和 exec 路径上获得内核身份。

系统对象	uCore 基础	AgentOS-uCore 增量
进程与线程	PID、内存、文件表、运行状态	Agent identity、role/capability、control id、workflow lifecycle
虚拟内存	页表、用户页与内核页	六页只读 Context mirror 与一页用户 cache
文件系统	inode、目录、读写与 fsync	boot-scoped metadata、选择性索引、typed live query
系统调用	参数拷贝、执行与返回	结构化工具目录、typed V2/V3、Task SQ/CQ
进程间通信	管道、信号与基本等待	MESSAGE route、watch/wait、heartbeat、LLM correlation
调度与资源	线程运行队列	workflow 实体、Credit Domain、EEVDF lag/deadline

表 1.1: uCore 基础与 AgentOS-uCore 增量

文件 metadata 使用当前 boot 的内存 sidecar，并与真实 dev+inum+incarnation 绑定。这种实现方式保留原 uCore 磁盘格式，也使索引、订阅和增量缺口恢复能够独立演进。

1.1.3. 机制与策略的分层

我们把稳定、有界、与系统效果直接相关的机制放入内核；将目标分解、历史组织和工具选择留在 Guest 用户态；将 TLS、API key 和 provider 协议放在 Host。这样的层次使确定性策略、strict replay 和真实模型共用同一套 Guest 与内核路径。

层次	主要对象	职责
Host	CLI、observer、provider adapter、QEMU serial	交互、传输、TLS 和 provider 格式转换
Guest 用户态	Coordinator、specialists、N1、artifact store	目标分解、任务状态机、工件组织和结果回灌
AgentOS 内核	identity、Context、tool、VFS、IPC、resource、scheduler	身份、权限、数据来源、等待、记账与调度

表 1.2: AgentOS-uCore 的三级职责

1.1.4. 相关机制与技术选择

资源记账参考 Linux cpuacct/rstat 的分层聚合思路 [2, 3]，运行记录参考 BPF ring buffer 的有序提交 [4]，调度使用 EEVDF 的 lag 与 virtual deadline 概念 [5]。Task Channel 采用 io_uring 的 SQ/CQ 通信形状 [6]，文件 metadata 和选择性索引参考 Haiku BFS[7]。Agent 工具循环中“模型选择、应用执行、结果回灌”的职责分配来自现有 tool-use 实践 [8, 9]。

这些选择在 uCore 中被重新组织为统一的 workflow 主体。identity、Context、VFS scope、Credit Domain 与 EEVDF 共用同一个 id+generation，从而避免各模块对 Agent 归属产生不同理解。

1.2. 设计目标与验收线索

项目将 Agent 的系统行为分解为主体、效果、记录与服务四类对象。主体由 identity 和 lifecycle 表示；效果由工具、文件和 IPC 的结构化请求表示；记录由 Context、provenance 与 evidence 表示；服务由 resource domain 与 workflow scheduler 表示。每类对象均有对应 Guest 程序和 Host 检查路径，性能实验只比较保持 workload/result fingerprint 一致的路径。

问题	内核增量	对应验证
角色与工作流不能被系统识别	受信映像、identity、control id、lifecycle generation	create/fork/exec/exit 与 stale generation 场景
工具调用参数与副作用分离	catalog、V2/V3、schema、contract、Task SQ/CQ	ABI、schema、retry/cancel、contract 场景
文件与消息缺少因果关系	Context Path、provenance、metadata、Evidence Ring	snapshot/branch、query/watch、fence 与 audit 场景
等待与资源以线程为单位碎片化	wait/wake、heartbeat、Credit Domain、Workflow EEVDF	竞争、唤醒、并发度与公平性测量

表 1.3: 从 Agent 负载问题到 AgentOS-uCore 内核机制

2. 系统设计

AgentOS-uCore 沿用 uCore 的进程、页表、VFS、trap、系统调用与调度主干，不以独立运行时替代内核对象。设计的关键是让同一个 workflow key 在身份、Context、文件、消息、资源和调度路径上保持可验证的一致性。图、数据结构与状态机共同约束各模块之间的交接点。

2.1. AgentOS-uCore 总体架构

2.1.1. 以 uCore 为底座的产品分层

AgentOS-uCore 是一组直接运行在 uCore 内核中的 Agent 功能模块。图2.1从下到上给出产品结构：uCore 提供基础系统对象，AgentOS 将六组核心机制嵌入这些对象，Console 与 Nexus 则使用统一 UAPI 组织单 Agent 和多 Agent 工作流。

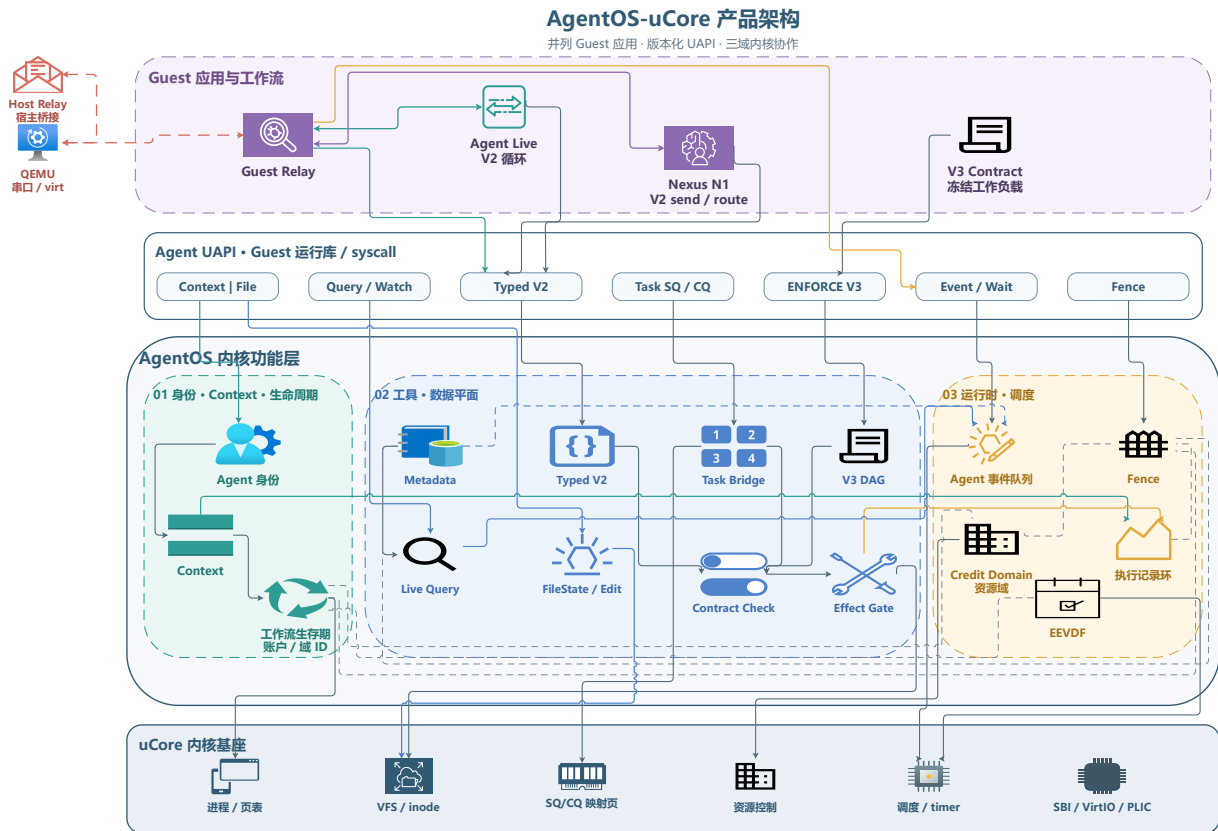


图 2.1: AgentOS-uCore 产品架构

底层 uCore 主干保持原有的进程、页表、VFS、trap、syscall 和 scheduler 责任。AgentOS 在其上形成三条相互配合的纵向路径：

1. 运行主体路径: identity 和 lifecycle 创建 Agent 主体, Context 保存每次工具交互, Credit Domain 与 EEVDF 负责资源和 CPU 服务。
2. 系统效果路径: typed tool 对调用进行 schema/capability 检查, VFS metadata 和 live query

处理文件对象，MESSAGE route 处理跨 Agent 通信。

- 3. 运行记录路径: Context Path 记录因果链, provenance 随文件、工具和 IPC 传播, Evidence Ring 与 Workflow Fence 产生工作流级截面。

2.1.2. 六项核心机制

模块	关键对象	主要功能
身份与生命周期	identity、control id、lifecycle	创建受信 Agent, 绑定 role/capability、workflow generation 与 VFS scope
结构化工具	catalog、V2/V3、SQ/CQ	发现工具, 检查 typed 参数, 执行适应式调用、冻结合同和有界任务提交
Context 与 Provenance	Context Path、mirror、ring	连接请求、结果、因果、分支和数据来源
Live-Query FS	metadata、index、watch	为显式登记文件提供属性、选择性查询、变化通知和增量缺口恢复
Agent Loop	event queue、route、heartbeat	提供原子 watch/wait、MESSAGE、LLM correlation、timeout 和 cancel
Workflow 资源与调度	Credit Domain、EEVDF entity	按 workflow 聚合记账, 使用 lag 和 virtual deadline 分配 CPU 服务

表 2.1: AgentOS 核心模块

六项机制通过统一 workflow id+generation 相互关联。一次工具调用同时关联调用者身份、Context predecessor、工具 schema、数据 provenance、资源账户与调度实体。workflow generation 使已经回收的 slot、watch 和 handle 不会命中新对象。

2.1.3. 一次 Agent 工具调用

一次完整调用从 Guest 读取工具目录开始。Guest 根据 role 构建可见工具集, 将模型结果编码为 typed request。请求进入内核后, AgentOS 依次完成 copyin 与 version/size 检查、identity/lifecycle 检查、tool resolve 与 schema decode, 再执行 contract、provenance、resource 和 effect 检查, 最后调用 VFS、IPC 或其他 uCore 子系统。返回前, Context Path 发布新记录, 下一轮可直接引用其 sequence。

当 Agent 等待模型、文件事件或其他角色时, agent_wait 将线程移入等待状态。事件入队与 waiter 安装使用原子顺序, 从而避免“查看队列为空后刚好错过唤醒”。

2.1.4. 上层运行时

Console 使用确定性策略连接身份、Context、文件查询、IPC 与恢复操作, 用于功能演示和配对测量。Nexus 在同一系统调用面上增加长驻会话、动态委派、工件组织和 provider 对接。两个运行时共用 AgentOS 内核对象, 可以在确定性测试和自然语言交互之间保持一致的系统语义。

图2.2从运行时序角度展示同一科研恢复任务的两种实现。Plain uCore 使用普通进程、文件与 IPC; AgentOS-uCore 在这条路径中加入可识别的 workflow 主体、结构化工具、选择性索引、事件等待和资源调度, 使每个阶段都能关联到明确的 Agent 与 workflow。

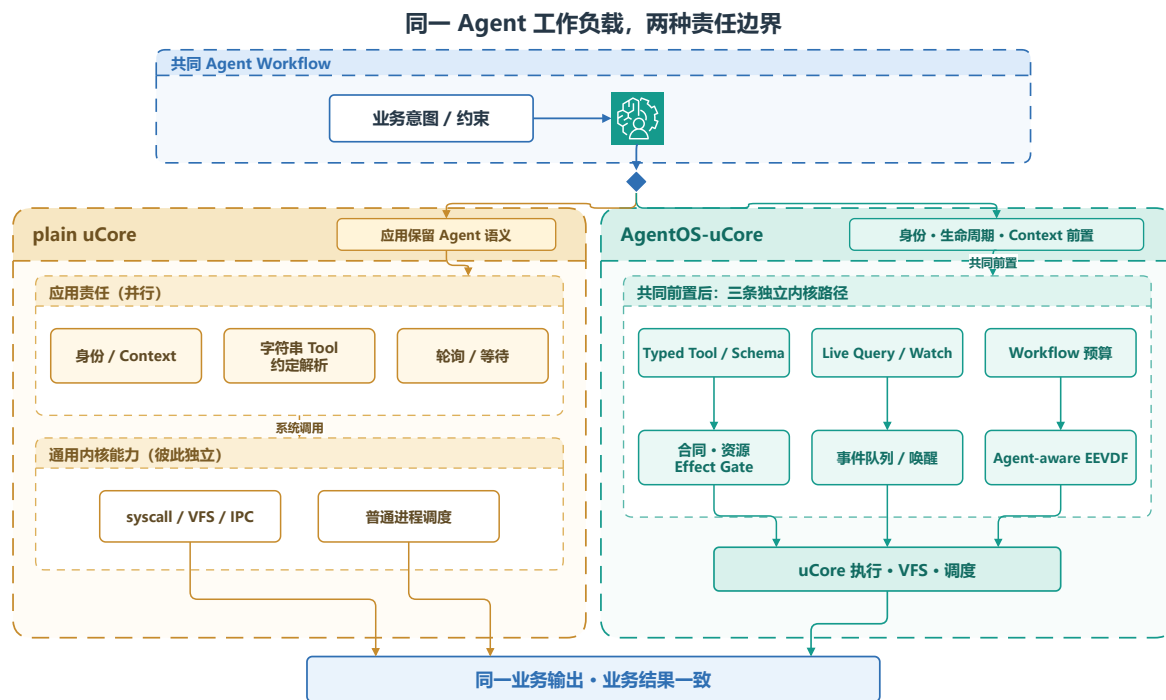


图 2.2: Plain uCore 与 AgentOS-uCore 内核路径

2.2. 跨模块状态约束

系统将 workflow 标识写为可复用 slot 的 id+generation。所有跨模块引用在使用前同时验证对象仍处于 active lifecycle、scope 一致且 generation 匹配；资源、metadata 和 evidence 采用 workflow key 归属。这样可以避免旧 slot 在关闭、回收、重新创建后被新的工作流错误接收。

Algorithm 1: 工作流键的跨模块校验

Input: 调用进程、目标对象、请求携带的 workflow key 与 scope。

Output: 可继续执行的对象引用，或明确的 stale/denied 结果。

1. 从调用进程取得内核保存的 lifecycle key 和 capability。
2. 检查 key 形状、lifecycle active 状态与 generation 是否匹配。
3. 比较调用者、目标对象和请求 scope；不一致时拒绝效果或事件传递。
4. 对工具、metadata、Task、evidence 或 scheduler 路径执行各自的参数与资源检查。
5. 仅在所有检查完成后发布状态、入队事件或提交资源。

3. 系统实现

实现部分以当前仓库的内核与用户态源码为依据。内核端的主要代码位于 `os/agent_*.c`、`os/workflow_*.c` 与 `os/resource_controller.c`；用户态场景位于 `user/src/`，Nexus 还使用 `user/lib/agent_nexus.c` 和 `host_tools/agentos_relayd.py`。以下各节先说明机制，再给出可直接定位的源代码片段。

实现域	主要源文件	关键责任
启动与身份	<code>os/main.c</code> 、 <code>os/agent_identity.c</code> 、 <code>os/workflow_lifecycle.c</code>	初始化 Agent 子系统，分配 <code>role</code> 、 <code>control id</code> 与 <code>workflow lifecycle</code> 。
上下文与执行	<code>os/agent_context.c</code> 、 <code>os/agent_core.c</code> 、 <code>os/agent_execution_contract.c</code>	发布 <code>Context</code> 、解析 <code>V2/V3</code> 请求、控制副作用合同。
任务与来源	<code>os/agent_task_channel.c</code> 、 <code>os/agent_provenance.c</code> 、 <code>os/agent_evidence_ring.c</code>	管理 <code>SQ/CQ</code> 、来源标签与 workflow 证据截面。
文件与事件	<code>os/agent_metadata_*.c</code> 、 <code>os/agent_live_query_events.c</code> 、 <code>os/agent_ipc.c</code>	维护 <code>catalog</code> 、 <code>query/watch</code> 、消息和等待。
资源与调度	<code>os/resource_controller.c</code> 、 <code>os/workflow_credit_domain.c</code> 、 <code>os/workflow_scheduler.c</code>	处理 <code>phase lease</code> 、 <code>credit</code> 、工作流级选择与记账。
集成场景	<code>user/src/labdemo_ucore.c</code> 、 <code>user/src/agent_nexus_ucore.c</code>	驱动恢复流程和四角色长驻协作。

表 3.1: AgentOS-uCore 实现源码分工

3.1. 内核原生 Agent 身份与生命周期

3.1.1. 从进程身份到 workflow 主体

AgentOS 需要区分普通进程、编排者和专职 Agent，同时让这些身份可以被 VFS、IPC、工具和调度器共用。我们将 Agent 身份作为内核状态安装到 PCB，并通过受信映像 `manifest` 和父角色授权创建。用户态程序可以读取 `identity`，权限判定始终使用内核副本。

受控 workflow 创建过程是一个完整事务：内核首先分配 `workflow lifecycle`，安装 `identity`、`role/capability` 和 `control id`，然后建立 `Context` 页、VFS scope 和资源账户。任一步骤失败都回收已分配对象。创建完成后，各子系统通过同一 `lifecycle key` 找到该 Agent 所属的工作流。

`agent_worker_create()` 不分配新的 workflow root lifecycle，而是在现有 lifecycle 下建立带受控 pending image 的子进程；它要求调用者在同一 workflow 内持有 ORCHESTRATE，并非 controller 专属权限。子进程仍须沿普通 `exec()` 路径装入该映像，不能把该调用理解为“已在新映像中运行”。`agent_scope_delegate_fd()` 只接受 `FD_INHERIT_DELEGATE`（当前用于 pipe），也不会立即把 fd 移入某个 scope：它在调用线程上设置一次性 ticket，只有下一次受控主体创建时的 spawn snapshot 会消费该 ticket；子进程以 `filedup()` 获得副本，父进程保留原 fd，随后 ticket 被清除。

3.1.2. 身份字段与角色策略

字段	含义	使用模块
<code>agent_id</code>	当前 boot 内的 Agent 编号	工具、Context、观测
<code>control_id</code>	内核分配的控制身份	controller 绑定、IPC route、handle 校验
<code>role/capability</code>	角色及其具体权限集	工具、文件、消息与工件效果
<code>workflow</code>	workflow 标识与可复用	lifecycle、watch、Task、resource、scheduler
<code>id+generation</code>	slot 的世代	
<code>VFS scope</code>	workflow 文件可见域	metadata、query、artifact
<code>exec/storage</code>	workflow 资源归属	进程、线程、文件、块、页与缓冲区
<code>account</code>		

表 3.2: Agent 身份核心字段

内核 role policy 将角色映射为 capability mask 和调度权重。Sentinel 主要读取进程、metadata 与调度状态；Investigator 获得受限内容读权限；Artifact 角色可以创建报告工件；Orchestrator 持有编排和角色创建权限。子 Agent 继承同一 workflow scope，其 capability 受父角色授权和映像 manifest 共同约束。

3.1.3. 七页 Context 地址区

每个 Agent 拥有固定 ABI 地址的七页 Context。前六页由内核维护，Guest 页表映射为只读，用于发布 identity header、最新返回、Context records、路径索引和稳定摘要。第七页是用户 cache，Guest 用于保存高频策略数据。identity、capability、scope 与因果序列均来自前六页的内核状态。

Context 固定容量为 128 条。每条记录包含 sequence、request/tool/status、service-start tick、cause、span、branch generation、control id 与记录摘要。内核使用 publish sequence 更新 mirror，Guest 也可以通过 query/snapshot UAPI 取得相同结构。

3.1.4. Lifecycle generation 与并发 gate

workflow slot 可在回收后复用，generation 每次重新分配。watch、资源账户、VFS sidecar、Task handle 与 artifact 都同时保存 id 和 generation。

lifecycle 使用 operation、departure 和 fence 三类 gate。普通系统调用进入 operation gate，exit/teardown 进入 departure gate，Workflow Fence 在两类操作排空后形成截面。当 controller 关闭且 member、active operation、departure 归零且 fence 已释放时，finalizer 清除该 lifecycle

的 Context、watch 与 ring。普通 VFS scope 完整回收；preserve_on_retire 持久输出 scope 在 storage account 为 DRAINING/FREE 时即可回收 lifecycle generation，但保留 registry、文件与 storage charge。

3.1.5. 资源主体随 workflow 创建

workflow admission 同时分配 exec 和 storage account。工作流内的进程、线程、文件对象、文件系统块/inode、Agent state page 和物理页均记录在同一账户下。这一设计使后续 Phase Lease 和 Workflow EEVDF 可以按工作流统一计算，子进程和多线程仍属于原工作流。

3.1.6. 验证结果

agenttrust_ucore 覆盖受信 exec、role grant、capability 衰减、fork/exec/exit 与普通进程共存；agentscope_ucore 覆盖 scope 与 generation 复用；agentfinal_ucore 覆盖 Context 固定映射、snapshot、branch、rollback 和容量淘汰。这些场景都在 RV64 uCore Guest 中执行真实系统调用。

3.2. 结构化工具执行

3.2.1. 工具目录与参数模式

Agent 对工具的需求同时包含“找到可用能力”和“在效果发生前校验调用”。AgentOS 内核维护 25 项工具目录，每项 descriptor 包含 version、size、tool id、name、schema、flags 与 side-effect 类别。启动阶段会检查 id/name 唯一性和 schema 完整性。

sys_tool_list 向 Guest 返回定长 descriptor。Guest runtime 再结合 role/capability 构建模型可见的工具集，补充用途、参数和结果说明。运行时内部的 wait、Context 与 LLM 原语保留给 Guest 自身，模型只看到当前角色可以请求的产品动作。

3.2.2. Exploratory typed V2

V2 request 携带显式 version、request size、tool name/id、parameter count 和 sized typed parameter 数组。当前 scalar 类型为 uint64 与 string，适合 uCore 中紧凑的参数解析。内核一次性拷贝参数，完成 key、type、数量和长度检查后，再进入共享执行器。

V2 适合自适应 Agent Loop。例如 Coordinator 可以先查询失败 stage，根据候选数决定是读取摘要还是创建 Research 子任务。每次调用独立检查 identity、capability、scope、schema、provenance 与资源状态。

3.2.3. 四种执行路径

Compact batch 用于同步小操作序列。V3 则在 workflow lifecycle 内冻结一份版本化合同，最多包含 24 个拓扑有序节点。每个节点关联 tool、schema digest、required capability、input/output artifact type、deadline、attempt、provenance/effect manifest 与 exec/storage envelope。

V3 request 同时绑定 contract generation、node/attempt、source node、predecessor Context sequence 和 32-byte fingerprint。成功或失败的 terminal 结果进入稳定完成槽，合法重试直接读取原结果，避免重复产生副作用。

路径	输入形式	主要用途	运行语义
Compact batch Typed V2	最多 64 个 agent_op typed key/value	固定顺序与 Context 压力场景 根据上一轮结果选择工具	一次 syscall 内顺序执行，逐项返回状态 逐次 schema/capability/scope 检查
ENFORCE V3	V2 prefix 与 contract envelope	可预先定义的高保证 DAG	冻结 node、predecessor、attempt、deadline 与 effect manifest
Task SQ/CQ	16-slot SQ 与 16-slot CQ	有界提交、完成与 backpressure	主线程 setup、issuer 绑定与世代校验；每个 accepted target 恰有一个 terminal CQE，cancel 不额外产生 CQE

表 3.3: 工具执行路径

3.2.4. Tool Phase Credit Lease

高风险工具可在执行前从 workflow account 锁定 exec/storage envelope。Phase Lease 依次经历 ADMITTED-→ACTIVE-→DEACTIVATED-→SETTLED。分配器在发布对象前获得 nonce claim，成功对象继续占有同一份 used credit，析构时完成一次结算。这一状态机适合多种对象在一次工具中共同达到峰值的场景。

3.2.5. Typed Task Channel

Task Channel 仅能由 Agent 主线程 setup，并映射 16-slot SQ 和 16-slot CQ，二者各占一个 Guest 映射页；复制的请求/终态与权威资源状态分别保存在两个内核私有页中。SQE/CQE 均为 128 字节。enter 与 resource binding 绑定 issuer tid 和 identity generation。内核先拷贝完整 SQE，再检查 enter ABI、递增 request id、contract/node/attempt、schema、deadline、cancel link 和 typed handle。issuer、owner 或 lifecycle 的 STALE 仅保留 ABI version/size 与 status，其余输出均置零；control generation 不匹配则经 enter result fill 返回当前 channel state。

内核私有 head/tail 是权威状态。CQ_FULL 只表示 completion 暂时没有可发布的槽位，是 backpressure 而不是 accepted target 的 terminal 状态；对应终态仍处于 pending，待有 CQ 槽后发布。SQE ring/slot generation 不一致或其他协议故障会使通道进入 sticky resync；issuer 必须以 AGENT_TASK_CHANNEL_ENTER_F_RESYNC 且空 sq_tail 显式复位，才能继续提交。每个 accepted target 在 success、denied、failure、cancel 或 timeout 后发布一个 terminal CQE。CANCEL 命令拥有独立 request id，只引用 target 而不另发 CQE；同步 policy denial 仍消费该 cancel id，并从 enter 返回 DENIED，目标继续保持原运行状态。target 已 ACK 或不存时同样消费 cancel id，enter result fill 返回当前 generation 与 waterlines。当前 provider 接受 null input 并返回 NONE，用于验证通道、截止时间、取消和完成语义。

3.2.6. 验证结果

agenttoolabi_ucore 覆盖目录发现、name/id 一致性、未知参数、类型错误和返回缓冲区；agentcontract_ucore 覆盖 DAG、deadline、retry/cancel、source Context 与 provenance

gate; agenttask_ucore 覆盖 SQ/CQ full、resync、cancel 和 terminal retention。一次性数据集进一步保存 96 条 16-op sequence，其分布在第 4 章集中分析。

3.3. Context Path、Provenance 与 Workflow Fence

3.3.1. 从多轮文本到结构化 Context

Agent 的下一步决策通常依赖上一次工具结果。AgentOS 在每次工具执行的效果位置建立 Context record，并由 sidecar 保存归一化的 agent_op/result 与归因；它不完整保留 V2 parameter、V3 binding 或 Task SQE，将请求、结果、因果和数据来源组织为可查询路径。

记录的 sequence 只在同一 clear epoch 内递增，字段包含 request/tool/status、service-start tick、cause sequence、span、branch generation、control id、短 payload/result 和 record hash；clear 后从 1 重新开始。path_parent_sequence 指向当前 active head，successor index 支持分支查询。

Context append 使用 prepare、fill、publish、commit 四个阶段；detail 在 record 标为 odd 前写入。mirror header 发布 count、head、oldest、latest、visible head、dropped 和 rollback count。Guest 在读取记录后重验 sequence/hash，再次确认 summary 未变，从而获得一致快照。

3.3.2. 分支、回退与淘汰

Context Path 保留 128 条记录。容量用尽后按 sequence 淘汰最早记录，同时更新 oldest/latest 和 dropped 计数。active path 只引用当前保留的 predecessor。

rollback 将 visible_head 移到一个仍然保留的 Context sequence，并为后续调用分配新 branch generation。已产生的文件、IPC 和 provider 效果保留原状。需要幂等的效果由 V3 attempt cache 与具体工具协议控制。这使 Context 分支可以专注于后续决策视图；manual note、rollback 与 clear 都不预留或提交 Evidence。

3.3.3. 数据来源传播

AgentOS 使用六类可组合 provenance 标签，标签随 Context、文件、工具返回、IPC 和 artifact 传播。

文件、工具、mailbox 和 Task resource 使用保守 OR 传播。摘要、重新哈希和跨 Agent materialize 会增加新来源，同时保留原标签。因此，文件数据可以作为模型观测，但不会自动成为控制授权。

3.3.4. 工具 Manifest 与效果检查

每个内核工具包含 safety manifest，其字段包括 accepted input labels、output-added labels、required capability 与 file/metadata/IPC/process/permission/artifact/watch side-effect mask。启用 ENFORCE V3 后，lifecycle generation、frozen contract edge、schema、capability、manifest 和 provenance 必须同时匹配。失败请求在工具效果前返回结构化状态。direct/V3 的拒绝通常记录 source Context、tool 与 reason；但 critical record 的预留或提交失败时，调用会以 NO_SPACE 或 RETRY 返回，因此不能把每一次拒绝都表述为必然持有 evidence ticket。

标签	含义	典型来源
KERNEL_FACT	内核结构化事实	PID、resource、scheduler、capability
TRUSTED_USER_CONTROL	经产品控制面提交的用户控制	新 turn、参数绑定审批
AGENT_DERIVED	Agent 从已有输入推导的数据	摘要、计划、报告
UNTRUSTED_FILE_DATA	来自文件的内容或 metadata	query、read、digest
UNTRUSTED_TOOL_OUTPUT	工具执行返回	typed V2/V3 result
CROSS_AGENT_DATA	经 IPC 或 artifact 跨角色传递	TASK result、broker artifact

表 3.4: Context provenance 标签

3.3.5. Evidence Ring

每个活跃 workflow 按需分配四个 Agent state page: 三页 ordinary ring, 共 48 个 256-byte slot; 一页 critical ring, 共 16 个 slot。direct/V3 的 canonical terminal/denial 路径预留递增 ticket, 填充后提交; manual note、rollback 与 clear 不走该预留/提交协议。当更早 ticket 仍在准备中时, 读者暂停在当前位置, 从而保持全局发布顺序。

普通成功 Context 写入 ordinary ring, 授权效果和成功建立记录的拒绝写入 critical ring。拒绝记录无法预留或提交时, direct/V3 会返回 NO_SPACE/RETRY, 而不是伪造 critical ticket。环形空间复用前, 已提交 segment 滚入内部 root。这套结构为 Workflow Fence 提供按 ticket 排序的运行记录。

3.3.6. Workflow Fence

controller 通过 agent_run(count=0, AGENT_RUN_F_FENCE) 发起 fence。内核检查 orchestrator/control 关系, 取得 lifecycle fence gate, 排空 metadata/live-query 延迟工作和 VFS reclaim, 紧缩 exec/storage credit 并确认 pending credit 归零, 最后密封 Evidence Ring。

非空的 fence request 只携带 version/struct_size、必须为零的 flags、必须为零的 reserved、challenge 和 request id, 不携带 previous root; request 与 receipt 可以同时为 NULL, 作为匿名 fire-and-forget fence 发起。previous root 是内核在完成截面后写入返回 receipt 的字段。提供 receipt 时, 形成的 320-byte receipt 绑定 previous root、challenge、fence sequence、ticket range、event/gap count、metadata generation、credit epoch、精确 used-credit digest 和 ordered segment digest。提供非零 request id 与 receipt 时, 内核在内部状态提交后 copyout; 同一 request id 的 copyout 重试返回已缓存结果。该 receipt 表达当前 boot 内的 workflow 截面, 其 flags 同时记录 metadata 的 volatile 性质与 coverage 状态。

3.3.7. 验证结果

agentfinal_ucore 连续执行 64 次工具调用, 然后覆盖 snapshot、branch、rollback 和窗口淘汰。provenance 场景将文件来源传入受保护副作用, 验证效果前拒绝与 critical ring 记录。fence 场景覆盖 challenge、retry cache、credit pending、metadata generation 和 copyout 重试。

3.4. Agent Live-Query 文件系统

3.4.1. Agent 对文件对象的结构化需求

科研和恢复 workflows 经常围绕“当前哪些对象满足条件”与“哪些对象刚刚发生变化”运行。目录遍历可以找到路径，工作流还需要 project、run、stage、status、kind、dependency 和内容摘要等属性。AgentOS 在 uCore VFS 之上增加 boot-scoped metadata catalog，将显式登记的文件组织为可查询、可订阅的 Agent 对象。

3.4.2. Metadata 与 inode incarnation

拥有 META_WRITE 的 Agent 调用 `agent_file_meta_set()` 设置 metadata，字段与真实 `dev+inum+incarnation` 绑定。更新使用 `update_mask`，删除使用显式 DELETE 动作。文件 create/rename 继续使用原 uCore VFS，只有 workflow 明确关心的对象进入 catalog。

incarnation 区分 inode number 复用前后的两个对象，并由 VFS 增量维护，随 query/watch 结果返回。query 或 watch 命中记录后，内核复核 catalog generation、lifecycle、scope 与完整 predicate。unlink、内容大小变化与对象失效会更新已登记 sidecar。

3.4.3. Traversal 与选择性索引

catalog 为 status/stage/kind 维护选择性索引。typed query 只携带 predicate 字段、query flags 与最大 hit 数；scope 由调用身份约束，实际 plan 由内核确定。结果返回 hits、candidate records、scanned records、total hits/returned/truncated、plan/plan reason 与 fs generation。调用者可用 SCAN 强制 traversal，或用 USE_INDEX 允许 planner 使用索引。启用索引时 planner 按固定的 status->stage->kind 次序取第一个已给定字段，并不比较基数；索引路径仍对候选记录执行完整 predicate 检查，保证 traversal/indexed 结果一致。最大返回数为 8，total_hits 仍报告全部命中数；当前 ABI 没有分页 cursor。

3.4.4. Typed live watch 与增量缺口恢复

`agent_live_watch()` 安装一份完整 `agent_file_query`。watch 保存的是 predicate、scope 与 generation，而不是一个持久结果集；每次 metadata 变化重新计算 before/after predicate：从未命中变为命中产生 ENTER，持续命中且记录变化产生 UPDATE，从命中变为未命中或对对象删除产生 LEAVE。事件通过 AGENT_EVENT_FILE_QUERY 进入目标 Agent 的 Context 和有界队列。

当队列或 pending 表无法保存全部增量时，内核递增 `resync_generation` 并发送 RESYNC_REQUIRED；正常连续的 fs generation 变化仍只发布 ENTER/UPDATE/LEAVE，不构成 resync。安全恢复保持旧 watch，先装同 query、无 ACK_RESYNC 的 replacement，再执行有界 query；仅 truncated=0 snapshot 是完整基线，随后才在旧 watch 上以对应 generation ACK_RESYNC 并 unwatch，replacement 覆盖切换窗口。单次 query 最多返回 8 hit、没有分页 cursor；snapshot 截断不能 rebase 或 ACK，须收紧 query，分页留待未来 ABI。resync 不以 Workflow Fence 为通用屏障。workflow close 仅使 lifecycle 进入 closing，不会 drain pending；Fence 排空 tombstone、content delta 并尝试投递 marker，只要该 workflow/scope domain 的 sticky resync 未由 ACK_RESYNC 确认，即使 marker 已投递仍返回 RETRY，最终 reclaim 清理。

3.4.5. 查询性能

实验使用 catalog size 24、64、96 和 exact hit count 1、2、4、8，组成 12 个实测网格。每格保留 15 个 AB/BA inner pair，同时记录 traversal/indexed latency、candidate count 和 records examined。

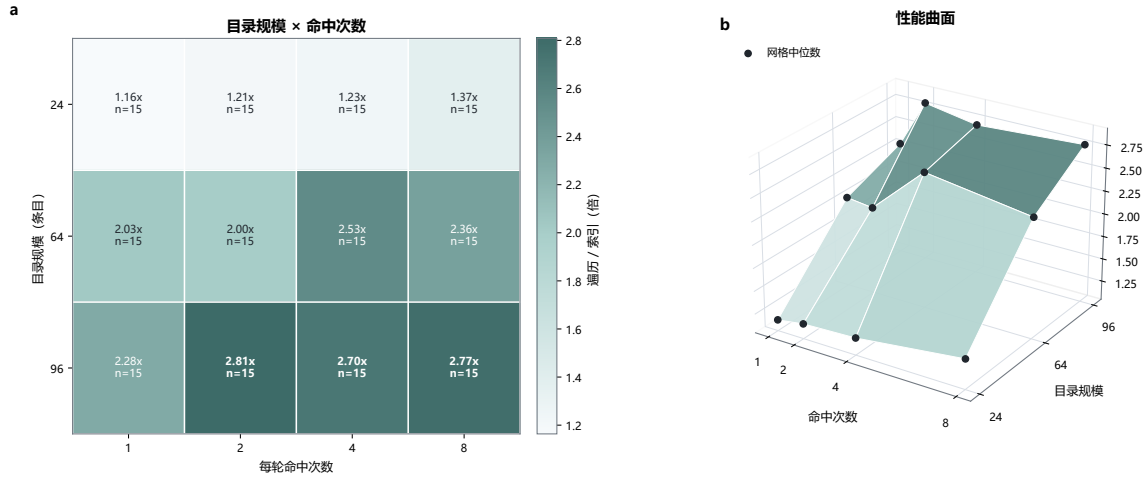


图 3.1: Catalog 查询加速比

图3.1的 12 个格点均来自实测，中位加速比范围为 1.164–2.808 倍。catalog 越大、hit 越集中时，选择性索引能够跳过更多无关记录。三维曲面用于连续展示这一趋势，热力图中的每个数字则对应原始网格样本。

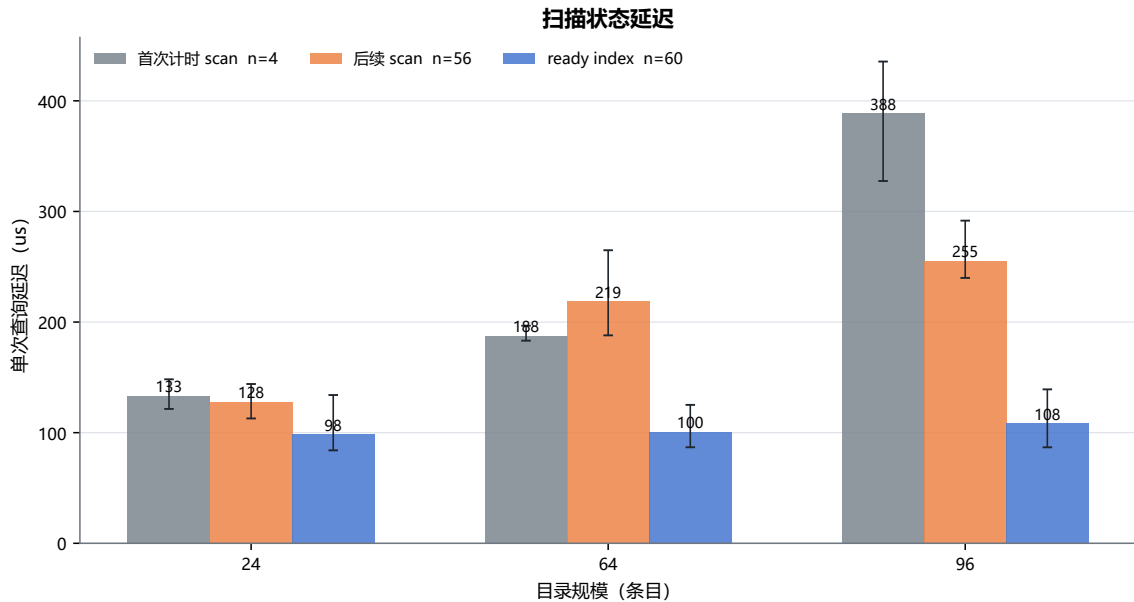


图 3.2: Traversal 与 Indexed 查询分组

图3.2将显式 warmup 后的第一次 timed scan、后续 repeat scan 和 ready index 分组展示。28 条 diagnostic 均记录 cache=ready，因此该图聚焦查询计划差异，而非宿主页缓存的冷启动效应。

3.4.6. 验证结果

`agentfs_ucore` 覆盖 `metadata set/update/delete`、`summary/digest`、`incarnation` 和 `scope`；`agentbench_ucore` 用同一数据集比较 `traversal/indexed` 的结果集、候选数与延迟。Live-query 场景覆盖 `ENTER/UPDATE/LEAVE`、`queue gap`、`RESYNC_REQUIRED` 与 `replacement-snapshot-ACK` 的安全交接。

3.5. Agent Loop 与 Workflow EEVDF

3.5.1. 从轮询循环到内核等待

Agent Loop 会等待用户输入、模型回复、文件变化、其他 Agent 的消息和 heartbeat。持续轮询会频繁进入内核，也会让大量等待线程占用调度时间。AgentOS 通过有界事件队列和 `watch/wait` 原语将等待转化为真实睡眠。

每个 Agent 拥有容量 16 的事件队列，其中保留 `kernel reserve`，并将同一 `source` 的排队事件限制为 4 条。`route` 表同样有固定容量。发送 `MESSAGE` 时，内核检查 `MESSAGE_SEND capability`、`active workflow`、`source/target route` 和 `watch`；公开 `agent_event` 只返回 `source/target PID`、`correlation` 等事件字段，`control_id` 与 `provenance/source label` 保存在内核 `sidecar`。

非 `TIMEOUT` 的 `agent_wait` 先检查已排队事件；选中事件时建立 `reservation`，无事件时以 `thread identity generation` 作为 `wait key` 安装 `waiter` 并使线程睡眠。事件入队与 `waiter` 状态使用相同锁序，确保到达事件能够对应到等待世代。`TIMEOUT` 路径在本地直接返回，不建立 `reservation`，也不写入 `Context`。`agent_wait_cancel()` 只是向 `wait` 路径注入 `AGENT_EVENT_CANCELLED`，并非工具或 `Task Channel` 的 `CANCEL` 操作；`event consume` 更新 `loop state` 与 `Context`。

3.5.2. Heartbeat 与 LLM correlation

Agent 可设置 `heartbeat interval`。`timer` 到期时，内核生成 `timer event` 并唤醒 Agent，Guest 根据当前状态决定检查文件、发送消息或再次等待。停止 `heartbeat` 时，`generation-bound timer` 与 `waiter` 一并清理。

`LLM_REQUEST/LLM_RESPONSE` 复用 `MESSAGE` 和 `wait substrate`。请求登记 `lifecycle`、`requester/relay` 的 `PID` 与 `control id`、`correlation` 和 `deadline`。`correlation` 对同一 `requester` 严格递增，成功入队后才更新。`Relay` 返回匹配 `response` 时，内核原子消费 `pending` 并生成 `LLM_DONE`。只有同一 `correlation` 仍处于 `active pending` 时才返回 `DUPLICATE`；已完成或已过期 `correlation` 的复用返回 `CONFLICT`。`pending` 使用 120 秒 `kernel-tick TTL`，`terminal history` 区分已消费与超时请求。

3.5.3. Workflow Credit Domain

资源账户维护 `used U`、`pending P`、`free F` 和 `held = U + P + F`。当前账户的可用 `credit` 先在本地状态中转移：

`reserve:F->P` `commit:P->U` `cancel:P->F` `release:U->F`

账户补充 credit 时检查 global、resource class 和 account hard limit。系统压力、context switch、account close 与 fence 会 trim F 。Workflow Fence 在同一锁中执行 trim+snapshot，要求 $P = 0$ ，并导出精确 U 、account generation、credit epoch 和向量摘要。

3.5.4. 按 workflow 聚合的 EEVDF

传统线程级公平会使多线程 workflow 获得更多调度候选位。AgentOS 将同一 resource domain 聚合为一个 workflow entity。调度器使用 workflow vruntime 计算 lag， $vruntime \leq global_vtime$ 的实体进入 eligible 集，再从中选择最早 virtual deadline。latency class 和 event/deadline 可以缩短 service request。一次 dispatch 完成后，内核按实际 service cycles 更新 workflow vruntime，睡眠实体的 lag 向 global vtime 衰减。

当只有一个 workflow entity 时，调度器走 fast path。实体表容量为 4，其中包含一个 BOOT_SEALED bootstrap participant，因此同时可纳入三个 fresh workflow。实体信息不完整或容量超出时，调度器使用原 uCore 路径并增加 fallback 计数。

3.5.5. 唤醒延迟与公平性

一次性实验使用 6 个 fresh QEMU boot，覆盖并发度 1、2、3、4 的 service window 和 exact wake probe。504 条 probe 直接读取 scheduler trace 中的 ready_age，其中 425 条为 0 tick，79 条为 1 tick。uCore tick 为 10 ms，这些样本表明该负载中的 fresh waiter 都在一个 tick 内获得调度。

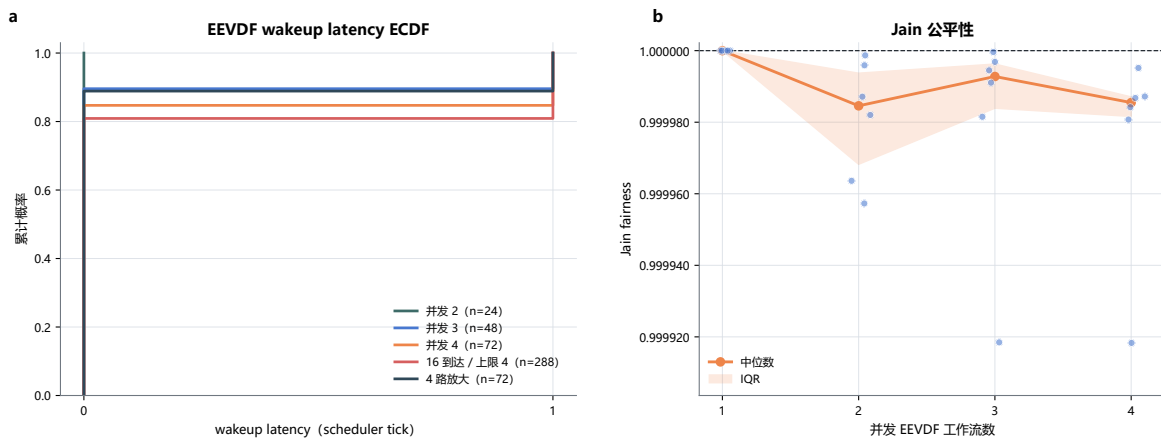


图 3.3: EEVDF 唤醒延迟与公平性

以同一 service window 的 raw cycles 计算 Jain fairness，并发 1/2/3/4 的中位数分别为 1.000000、0.999985、0.999993 和 0.999985。图 3.3 的 ECDF 保留了每个唤醒探针，折线则展示不同并发度下工作流服务量的均衡程度。

3.5.6. 验证结果

agentloop_ucore 覆盖原子 wait/wakeup、timeout、cancel、heartbeat 和 MESSAGE route；agentsched_ucore 与 agent_eevdf_ucore 覆盖 fast path、实体容量、线程放大、sleep decay、fallback、wake histogram 和 Jain fairness。

3.6. AgentOS Console 综合 workflow

3.6.1. 科研恢复场景

AgentOS Console 将六组内核机制组织成一个确定性科研恢复 workflow。labdemo_ucore 创建 Orchestrator、Sentinel、Investigator 和 Recovery，建立 workflow identity、MESSAGE route、Context、metadata 与资源账户。场景登记 prepare/align/analyze/report/archive 五个 stage 及其依赖，然后查询失败节点、读取摘要、跨 Agent 发送恢复请求，完成 align 恢复与 report 更新。

确定性 policy 固定每个阶段的业务工作量，工具结果仍由 AgentOS 内核执行和检查。这样既能连接身份、Context、Live-Query、IPC wait 与恢复写入，也能在不同实现路径间建立等量比较。

3.6.2. Traversal 与 Indexed 配对

综合场景保留 traversal/compat 和 indexed/native 两条功能等价路径。每个独立 QEMU boot 内按 A/B 或 B/A 顺序运行，两侧输出相同 workload/result fingerprint。核心窗口覆盖查询、恢复写入、fsync 和复核，query block 另行记录纯查询区间。

规范数据集包含 16 个独立 workflow boot，AB/BA 各半。16/16 个 core interval 中 indexed 延迟更低，traversal/indexed core 中位数分别为 34.713 ms 和 13.294 ms，配对加速比中位数为 3.118 倍。

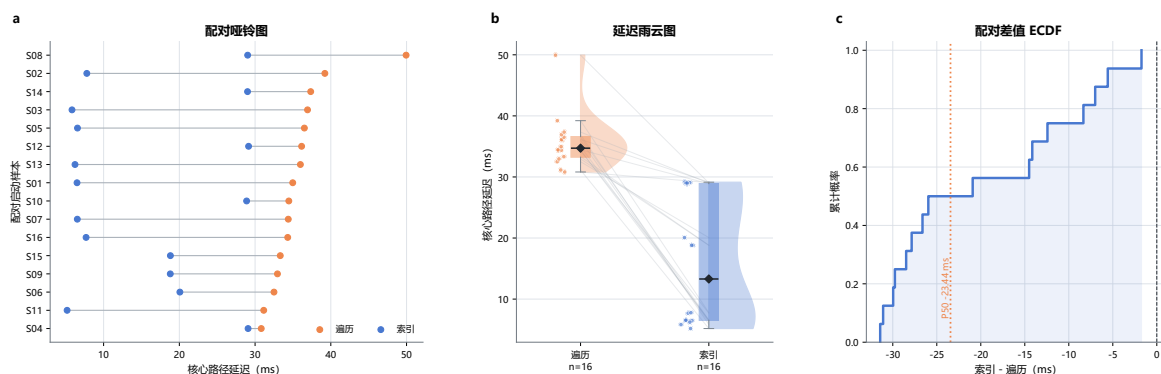


图 3.4: Traversal 与 Indexed Core 配对性能

图3.4的哑铃图逐 boot 连接两条路径，右侧分布图给出各自样本形状，ECDF 则直接展示配对差值。所有配对均落在 indexed 更低的一侧，且保留了不同 boot 的波动。

3.6.3. Core 与端到端窗口

核心窗口用于观察 AgentOS 数据路径，端到端窗口还包含场景启动、恢复编排、fsync、复核和统计输出。在 E2E 窗口中，indexed 于 3/16 个配对取得更低延迟，indexed-traversal 中位数为 +13.452 ms。这一结果表明，当前整体时间主要受恢复写入、fsync 与复核阶段影响，索引带来的 core 收益还需与这些阶段联合优化。

AB 与 BA 两个顺序分层的 core speedup 中位数分别为 1.4535 倍和 5.4805 倍。顺序差异使配对设计显得尤其重要：每个 boot 保留双路径原始时间，汇总时再分别展示 AB 和 BA。

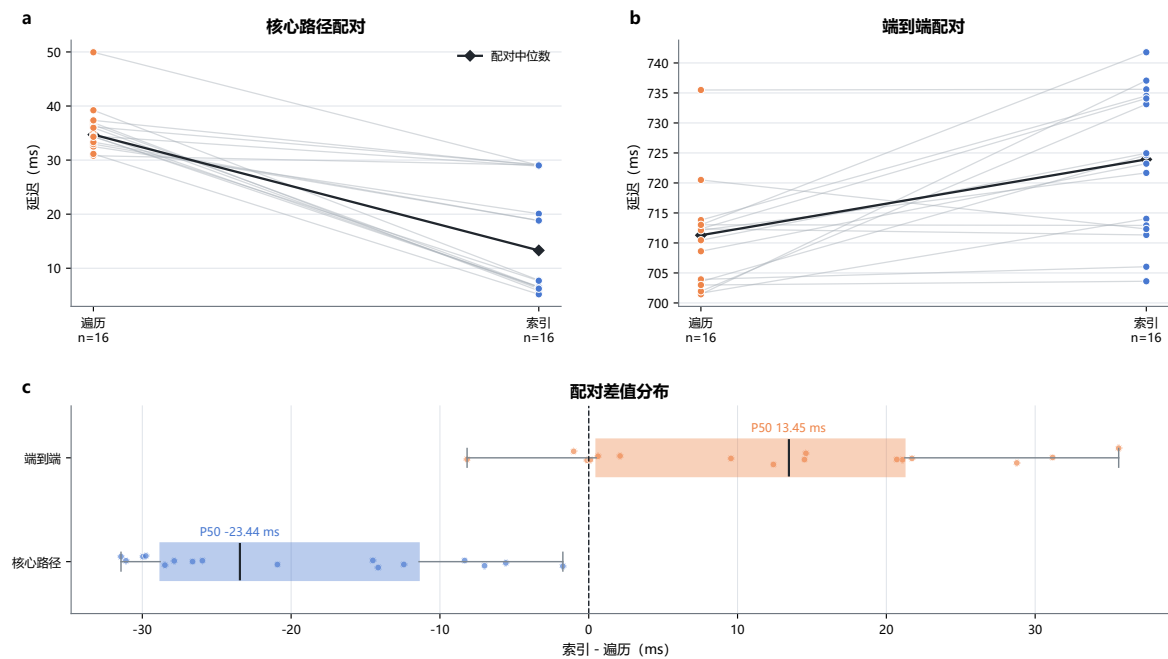


图 3.5: Core 与端到端窗口

3.6.4. 系统结果

综合工作流串起 Agent 创建、Context、typed tool、Live-Query、MESSAGE wait、恢复写入和运行记录。输出包含业务 fingerprint、每阶段状态、records examined、核心/E2E 时间和工作量计数。双目标路径另行构建 Plain uCore 与 AgentOS-uCore，在相同用户态合同下比较系统实现。

3.7. AgentOS Nexus 多智能体协作平台

3.7.1. 长驻会话与四角色协作

AgentOS Nexus 是建立在 AgentOS-uCore 之上的交互运行时。一个 Nexus session 在同一 workflow 中长驻 Coordinator、System、Research 和 Analyst 四个业务 Agent，每个角色都拥有独立 PID、agent id、control id、role/capability 与 Context。四个角色共享 workflow lifecycle、VFS scope、resource domain 与 EEVDF entity，以工作流作为资源和调度主体。

产品角色	内核 role	职责	主要 capability
Coordinator	Orchestrator	维护会话、调用模型、创建子任务、验证返回	编排、受控工具、工件 broker
System	Sentinel	读取当前 boot 的进程、Context、文件大小和调度状态	metadata/process/scheduler read
Research	Investigator	读取已登记的本地资料与历史测量工件	content read、query、digest
Analyst	Artifact	合并 System/Research 结果并生成报告	artifact read/write、report publish

表 3.5: Nexus 业务角色

另有一个 Relay Agent 负责 QEMU 串口 frame。Host daemon 持有本地 socket、TLS、API key 与 provider JSON adapter，Guest Relay 将 model、controller 和 telemetry frame 分流。业务文件与 artifact 保存在 Guest VFS，Coordinator 依据真实工具返回决定下一步委派。

3.7.2. 一次交互回合

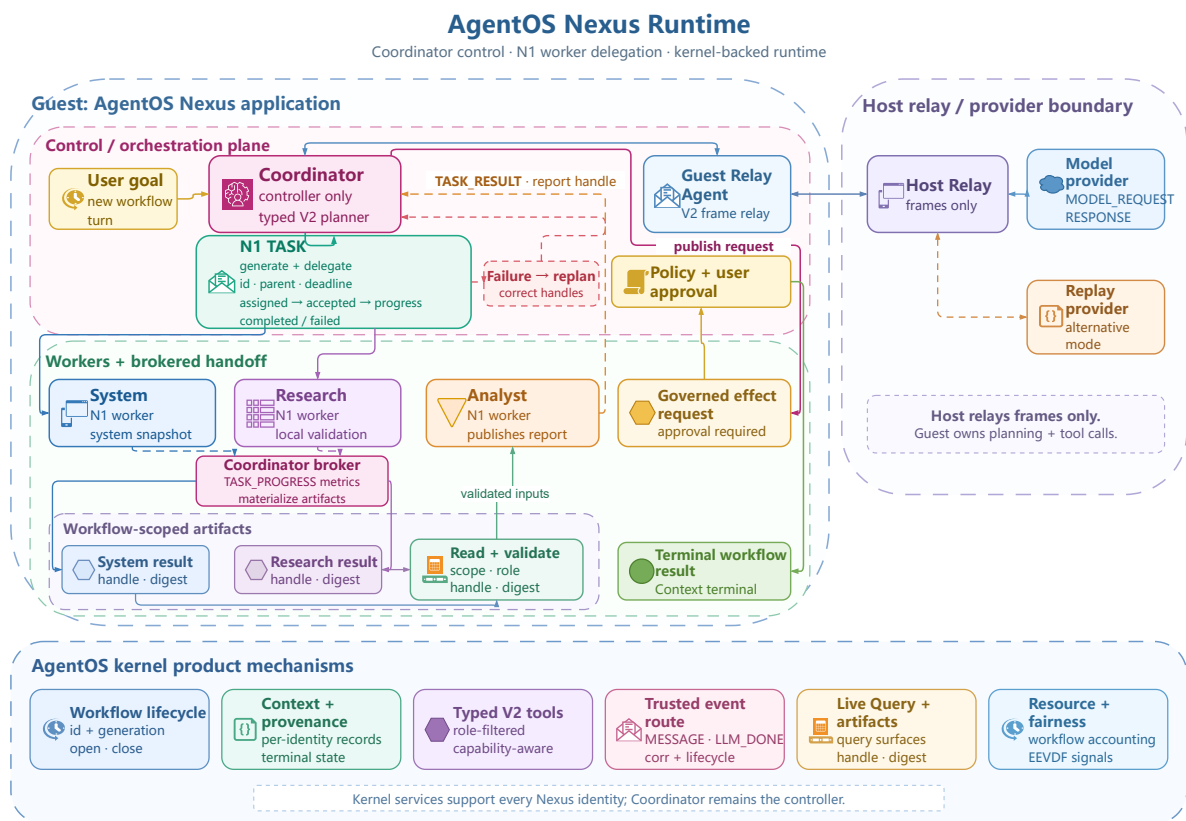


图 3.6: Nexus 四角色委派与三平面运行时

图3.6展示一次从用户目标到最终结果的运行流程。

1. CLI 将新目标作为 turn 提交给 Coordinator。
2. Coordinator 组织有界历史和 role-filtered 工具集，发出 LLM_REQUEST 后进入 WAITING_LLM。
3. Relay 将请求交给 live provider 或 strict replay，响应到达时由 AgentOS 唤醒 Coordinator。
4. 模型需要专门信息时，Coordinator 使用 N1 创建 System、Research 或 Analyst 子任务。
5. Worker 以自己的 identity、Context 和 capability 执行 typed V2 工具，等待期间进入 WAITING_EVENT。
6. System 和 Research 以 TASK_PROGRESS metrics 回传 Coordinator；由 Coordinator materialize 较长结果为 workflow-scoped artifact。Analyst 自身发布 report artifact，并以 TASK_RESULT 返回 report handle；Coordinator 取得该 handle 后发起 governed publish effect。

7. 工具结果、结构化失败和审批结果都返回模型，使后续一轮能够继续执行或重新规划。

3.7.3. N1 子任务协议

N1 是 Guest 用户态的 44-byte canonical little-endian envelope，以 N1：加无填充 base64url 通过 typed V2 `send_message` 传输。每条任务包含 lifecycle id/generation、task id、parent task id、source/target、deadline、state 和有界 value。状态机包含 TASK_ASSIGN、TASK_ACCEPT、TASK_PROGRESS、TASK_RESULT、TASK_FAILED 和 TASK_CANCEL。

AgentOS 内核对 MESSAGE 层的发送者、接收者、route、scope、provenance、队列和唤醒进行检查，Nexus runtime 处理 N1 decode、transition validation、deadline 和单项在途。较长的 objective、工具结果和报告使用 artifact handle 引用，使控制消息保持紧凑。

3.7.4. Artifact 数据面

Artifact manifest 绑定 lifecycle、handle generation、kind、task/parent、producer、owner、materializer、permission mask、payload size、source 和 provenance。Guest 以流式 SHA-256 覆盖完整 payload，读取者同时校验 manifest 和 digest。

System 和 Research 不在任务终态直接发布 artifact，而是以 TASK_PROGRESS metrics 回传 Coordinator；Coordinator 以 broker 方式 materialize 对应 artifact，并保留 logical producer 与 materializer。Analyst 读取两份上游工件后以自身身份发布 report kind，并以 TASK_RESULT 返回该 report handle；Coordinator 取得 handle 后才发起 governed publish effect。provenance 取各输入的并集，因此报告可以同时保留 file、tool 和 cross-agent 来源。

3.7.5. 动态工具呈现

Nexus 启动时从 AgentOS 内核读取 25 项工具目录，检查 name/ID/schema，再按 product role、kernel role、capability 和 side-effect class 筛选。tool_search 按类别分页返回 rich description。

默认读取面包含 pid_info、ctx_stat、query_process、get_system_status、read_context、query_file、read_file_summary、read_file_digest、dependency_query 和 read_message。delegate_task、read_artifact 与 publish_report 是 Nexus 产品动作，由 Coordinator 组织成 N1、artifact 与 typed V2 操作。

会话保留最近 4 个完整 turn，每个 turn 最多 8 个 model round，更早内容进入结构化 summary。每一轮模型请求都包含当前可见 schema 与上一次稳定工具结果，使工具输出直接改变后续决策。

3.7.6. 调用级审批与取消

Coordinator 取得 report handle 后，publish_report 的审批对象包含 session、turn、request、correlation、tool id、canonical arguments digest、一次性 nonce、issued tick 和 expiry tick。Coordinator 侧 Guest 收到决定后逐字段比较，通过时再执行精确 selector 的 typed V2 artifact_update。strict replay 包含一次拒绝发布，报告 artifact 保持可读，会话继续返回最终结果。

Ctrl-C 取消当前 turn，长驻 Agent 和 QEMU 继续运行；/quit 执行 session close。关闭顺序为停止 observer producer、排空内核观测事件、join observer、关闭 telemetry writer，Relay 在 EOF 后输出 SESSION_CLOSED。

3.7.7. 内核观测面

Nexus observer 从 agent_timeline_wait/read、agent_audit_query、agent_info 和 Context snapshot 生成两类投影。kernel_audit 显示 MESSAGE enqueue/consume 的 lifecycle、identity、source/target、correlation 和 status；kernel_snapshot 显示四个业务 Agent 的 Context sequence、wait/wakeup delta、scheduler dispatch/budget/vruntime 和 capability mask。

controller 在子任务分配和 worker reply 到达时主动 drain 共享 audit cursor，observer thread 使用同一 mutex 访问 cursor。左窗口展示交互与工具结果，右窗口同步显示 AgentOS 对应的消息、等待、唤醒和调度状态。

3.7.8. 运行验证

自动验收使用 strict replay。每条 fixture response 与 Guest 当次产生的 canonical MODEL_REQUEST SHA-256 绑定，并在当前 RV64 镜像的真实 QEMU boot 中执行。黄金场景包含 3 个连续 turn、4 个业务 identity、System 快照、Research stale-handle 失败后重新规划、Analyst 合并报告、一次审批拒绝和完整 session close。live 模式复用同一 Guest、N1、artifact 和 typed tool 路径，在 Host provider adapter 中对接实时模型服务。

3.8. 关键实现证据

本节将设计章节中的对象关系落实到当前代码树。代码框保留文件相对位置与物理行号，便于从文档直接回到内核实现核对。选取的片段覆盖启动、身份、调用、合同、上下文、文件事件、任务通道、资源与调度；它们共同构成一次 Agent 工作流从创建到完成的可执行路径。

3.8.1. 启动、身份与工作流世代

内核入口在建立进程、控制台和页表等基础组件后调用 agentinit()，并在文件系统准备完成后启用 Agent storage 和文件 epoch 运行时。这样，后续加载的初始用户程序能够在已初始化的生命周期、metadata 与资源子系统上运行，而不是由用户态临时拼装一套旁路状态。

Code 1 : uCore 启动序列中的 AgentOS 初始化

c

```

19 void main()
20 {
21     clean_bss();
22     printf("hello world!\n");
23     proc_init();
24     console_init();
25     agentinit();
26     kinit();

```

```

27     kvm_init();
28     if (kalloc_physical_policy_init(PHYSICAL_PAGE_SYSTEM_RESERVE,
29                                     PHYSICAL_PAGE_ORDINARY_LIMIT) < 0)
30         panic("physical page policy");
31     trap_init();
32     plicinit();
33     virtio_disk_init();
34     binit();
35     fsinit();
36     timer_init();
37     agent_storage_init();
38     fs_epoch_runtime_enable();
39     load_init_app();
40     infof("start scheduler!");
41     /* 仅启动期使用的映像加载完成后，才开放运行时输入输出准入。 */
42     bio_policy_start();
43     virtio_disk_runtime_start();
44     scheduler();

```

生命周期表使用 `id+generation` 作为查找键；`slot` 复用时 `generation` 单调推进。创建时先准备 `record`，再将 `owner`、`scope`、成员数和 `generation` 一次写入。因而持有旧 `generation` 的 `watch`、`Task` 或资源句柄都会在下一轮检查时失效，而不会误指向新工作流。

Code 2：工作流生命周期的创建与世代绑定

c

```

56 static struct workflow_lifecycle_record *
57 workflow_lifecycle_find_locked(struct workflow_lifecycle_key key)
58 {
59     uint slot;
60     struct workflow_lifecycle_record *record;
61
62     if (!workflow_lifecycle_key_valid(key) ||
63         key.id > WORKFLOW_LIFECYCLE_CAP)
64         return 0;
65     slot = key.id - 1;
66     record = &workflow_lifecycles[slot];
67     if (!record->used || record->generation != key.generation)
68         return 0;
69     return record;
70 }
71
72 static struct workflow_lifecycle_key
73 workflow_lifecycle_record_key(uint slot,
74                               const struct workflow_lifecycle_record *record)
75 {
76     struct workflow_lifecycle_key key = {
77         .id = slot + 1,
78         .generation = record->generation,
79     };
80
81     return key;
82 }
83
84 int workflow_lifecycle_create(uint scope_id,
85                               struct workflow_lifecycle_key *key)

```

```

86 {
87     int enabled;
88     int result = -1;
89     uint allocated_slot = WORKFLOW_LIFECYCLE_CAP;
90     uint64 next_generation = 0;
91
92     if (key == 0 || scope_id < VFS_SCOPE_FIRST_DYNAMIC ||
93         scope_id >= FS_OWNER_SCOPE_FLAG)
94         return -1;
95     *key = workflow_lifecycle_none();
96     enabled = intr_save();
97     for (uint i = 0; i < WORKFLOW_LIFECYCLE_CAP; i++)
98         if (workflow_lifecycles[i].used &&
99             workflow_lifecycles[i].scope_id == scope_id)
100             goto out;
101     for (uint i = 0; i < WORKFLOW_LIFECYCLE_CAP; i++) {
102         struct workflow_lifecycle_record *record =
103             &workflow_lifecycles[i];
104         uint64 generation;
105
106         if (record->used ||
107             workflow_lifecycle_generations[i] == ~0ULL)
108             continue;
109         generation = workflow_lifecycle_generations[i] + 1;
110         if (generation == 0 ||
111             !agent_identity_lease_lifecycle_contains(i, generation))
112             continue;
113         workflow_lifecycle_generations[i] = generation;
114         record->used = 1;
115         record->scope_id = scope_id;
116         record->generation = generation;
117         record->controller_control_id = 0;
118         record->next_context_branch = 0;
119         record->fence_sequence = 0;
120         record->members = 1;
121         record->active_operations = 0;
122         record->departing_operations = 0;
123         record->fence_gate = 0;
124         record->closing = 0;
125         *key = workflow_lifecycle_record_key(i, record);
126         allocated_slot = i;
127         next_generation = generation == ~0ULL ? 0 : generation + 1;
128         result = 0;
129         break;
130     }
131 out:
132     intr_restore(enabled);
133     if (allocated_slot < WORKFLOW_LIFECYCLE_CAP)
134         agent_identity_lease_lifecycle_note_next(
135             allocated_slot, next_generation);
136     return result;

```

角色策略不是用户态字符串约定。内核以 role 查表取得 capability mask 和调度权重，再在授权入口中检查父角色、目标角色与 capability 的合法关系。由此，工具、VFS、IPC 与 scheduler 可以共同消费同一份身份事实。

Code 3：角色策略表查询与调度权重取得

c

```

128 const struct agent_role_policy *
129 agent_identity_role_policy(int role)
130 {
131     for (uint i = 0; i < NELEM(role_policies); i++)
132         if (role_policies[i].role == role)
133             return &role_policies[i];
134     return 0;
135 }
136
137 int
138 agent_identity_role_valid(int role)
139 {
140     return agent_identity_role_policy(role) != 0;
141 }
142
143 int
144 agent_identity_role_sched_weight(int role)
145 {
146     const struct agent_role_policy *policy =
147         agent_identity_role_policy(role);
148
149     return policy ? policy->sched_weight : 50;
150 }

```

Code 4：父角色授权与进程 scope 校验

c

```

190 int
191 agent_identity_authority_check(struct proc *p, int role)
192 {
193     if (!agent_identity_role_valid(role))
194         return AGENT_STATUS_BAD_PARAM;
195     if (p == 0 ||
196         (p->is_agent ?
197          (!exec_policy_process_allows_role(p, p->agent_role) ||
198           !exec_policy_process_allows_role(p, role)) :
199          (!exec_policy_process_bootstrap(p) ||
200           !exec_policy_process_allows_role(p, role))) ||
201         (p->agent_role_grant_mask & AGENT_ROLE_GRANT_BIT(role)) == 0)
202         return AGENT_STATUS_DENIED;
203     return AGENT_STATUS_OK;
204 }
205
206 uint
207 agent_identity_proc_scope(struct proc *p)
208 {
209     if (p == 0 || !p->is_agent ||
210         p->agent_control_state != AGENT_CONTROL_OPEN ||
211         p->vfs_scope_id < VFS_SCOPE_FIRST_DYNAMIC ||
212         p->vfs_scope_id >= FS_OWNER_SCOPE_FLAG ||
213         !vfs_proc_lifecycle_active(p) ||
214         !vfs_scope_active(p->vfs_scope_id))
215         return VFS_SCOPE_NONE;
216     return p->vfs_scope_id;

```

Algorithm 2 : 带 generation 的工作流进入检查

1. 从当前 PCB 取得 workflow key，并验证其 id 与 generation 非零。
2. 在生命周期表中按二元键查找：只以 slot 相同不能通过校验。
3. 在 operation gate 加一后执行系统效果：fence 或 teardown 已关闭时返回 retry/stale。
4. 所有异步对象在完成、取消或回收时再次比较 generation，再释放生命周期引用。

3.8.2. 结构化工具调用与执行合同

V3 调用首先执行用户地址范围、ABI version/size、保留字段、参数个数与 lifecycle 绑定检查。请求中携带的 contract、node、attempt、schema digest 与输入指纹被复制至内核 binding；这使一次工具调用不是一个孤立的函数调用，而是合同 DAG 中可验证的节点执行。

Code 5 : V3 请求的 ABI、生命周期和合同绑定检查

c

```

2590     if (copyin(p->pagetable, (char *)&req, reqaddr, sizeof(req)) < 0 ||
2591         user_range_check(p->pagetable, respaddr, sizeof(resp), PTE_W) < 0)
2592         return -1;
2593     memset(&resp, 0, sizeof(resp));
2594     resp.version = AGENT_CALL_VERSION_V3;
2595     resp.size = sizeof(resp);
2596     resp.request_id = req.request_id;
2597     lifecycle = vfs_proc_lifecycle(p);
2598     resp.contract.lifecycle.id = lifecycle.id;
2599     resp.contract.lifecycle.reserved = 0;
2600     resp.contract.lifecycle.generation = lifecycle.generation;
2601     resp.contract.generation =
2602         agent_execution_contract_generation(lifecycle);
2603     resp.node_id = req.node_id;
2604     resp.attempt_id = req.attempt_id;
2605     memmove(resp.input_fingerprint, req.input_fingerprint,
2606             sizeof(resp.input_fingerprint));
2607     resp.source_context_sequence = req.source_context_sequence;
2608     memset(&binding, 0, sizeof(binding));
2609     binding.lifecycle.id = req.contract.lifecycle.id;
2610     binding.lifecycle.generation = req.contract.lifecycle.generation;
2611     binding.contract_generation = req.contract.generation;
2612     binding.node_id = req.node_id;
2613     binding.attempt_id = req.attempt_id;
2614     memmove(binding.input_fingerprint, req.input_fingerprint,
2615             sizeof(binding.input_fingerprint));
2616     binding.source_context_sequence = req.source_context_sequence;
2617     binding.source_control_id = req.source_control_id;
2618     binding.source_pid = req.source_pid;
2619     binding.source_reserved = req.source_reserved;
2620     memmove(binding.schema_digest, req.schema_digest,
2621             sizeof(binding.schema_digest));
2622     binding.input_artifact_type = req.input_artifact_type;
2623     binding.source_node_id = req.source_node_id;
2624     binding.input_mode = AGENT_EXECUTION_INPUT_INLINE;
2625     memset(&op, 0, sizeof(op));
2626     op.version = AGENT_CALL_VERSION_V3;
2627     op.tool_id = req.tool_id > 0 ? req.tool_id :

```

```

2628     (0x20000000 | (int)((req.node_id + 1) & 0xffffU));
2629     op.request_id = req.request_id;
2630     op.arg0 = req.contract.generation;
2631     op.arg1 = ((uint64)req.node_id << 32) | req.attempt_id;
2632     safestrcpy(op.payload, "v3_rejected", sizeof(op.payload));
2633     enforced = p->is_agent &&
2634         agent_execution_contract_enforced(lifecycle);
2635     memset(error, 0, sizeof(error));
2636     if (req.version != AGENT_CALL_VERSION_V3) {
2637         status = AGENT_STATUS_BAD_VERSION;
2638         safestrcpy(error, "bad_request_version", sizeof(error));
2639     } else if (req.size != sizeof(req)) {
2640         status = AGENT_STATUS_BAD_SIZE;
2641         safestrcpy(error, "bad_request_size", sizeof(error));
2642     } else if (req.flags != 0 || req.source_reserved != 0) {
2643         status = AGENT_STATUS_BAD_PARAM;
2644         safestrcpy(error, "unsupported_fields", sizeof(error));
2645     } else if (req.param_count > AGENT_TOOL_PARAM_MAX) {
2646         status = AGENT_STATUS_BAD_SIZE;
2647         safestrcpy(error, "too_many_params", sizeof(error));
2648     } else if (req.contract.lifecycle.id == 0 ||
2649         req.contract.lifecycle.generation == 0 ||
2650         req.contract.generation == 0) {
2651         status = AGENT_STATUS_BAD_PARAM;
2652         safestrcpy(error, "bad_contract_binding", sizeof(error));
2653     } else if (!p->is_agent) {
2654         status = AGENT_STATUS_NOT_AGENT;
2655         safestrcpy(error, "not_agent", sizeof(error));
2656     } else if (req.contract.lifecycle.reserved != 0 ||
2657         req.contract.lifecycle.id != lifecycle.id ||
2658         req.contract.lifecycle.generation != lifecycle.generation) {
2659         status = AGENT_STATUS_STALE;
2660         denial_reason = AGENT_EXECUTION_REASON_STALE_LIFECYCLE;
2661         safestrcpy(error, "stale_lifecycle", sizeof(error));
2662     } else {
2663         status = agent_tool_protocol_resolve(req.tool_id, req.tool_name,
2664             &tool, error, sizeof(error));
2665         if (status != AGENT_STATUS_OK)
2666             denial_reason = AGENT_EXECUTION_REASON_TOOL_MISMATCH;
2667     }

```

解析阶段先对工具名或 id resolve，再逐项 copyin typed parameter。每个参数必须匹配目录规则中的 key、类型、大小和唯一性；最后还要验证所有 required 参数均已出现。用户态即使构造了形式正确的结构，也不能绕过 kernel catalog 的 schema。

Code 6 : V2 参数 schema 解码器

c

```

548 int agent_tool_protocol_decode_v2(pagetable_t pagetable,
549     struct agent_request_v2 *request,
550     struct agent_tool_match *match,
551     struct agent_op *op, char *error,
552     int error_size)
553 {
554     uint seen = 0, count = rule_count(match->tool_id);
555     const struct param_rule *tool_rules = rules_for_tool(match->tool_id);

```

```

556
557     memset(op, 0, sizeof(*op));
558     op->version = AGENT_CALL_VERSION_V1;
559     op->tool_id = match->tool_id;
560     op->request_id = request->request_id;
561     for (uint i = 0; i < request->param_count; i++) {
562         struct agent_param_v2 param;
563         uint text_length = 0;
564         int index = -1;
565
566         if (copyin(pagetable, (char *)&param,
567             request->params + (uint64)i * sizeof(param),
568             sizeof(param)) < 0)
569             return -1;
570         if (param.version != AGENT_PARAM_VERSION)
571             REJECT(AGENT_STATUS_BAD_VERSION, "bad_param_version");
572         if (param.size != sizeof(param) ||
573             string_length(param.key, AGENT_PARAM_KEY_SIZE, 0) < 0)
574             REJECT(AGENT_STATUS_BAD_SIZE, "bad_param_size");
575         for (uint j = 0; j < count; j++)
576             if (!strncmp(param.key, rule_key(&tool_rules[j]),
577                 AGENT_PARAM_KEY_SIZE)) {
578                 index = j;
579                 break;
580             }
581         if (index < 0)
582             REJECT(AGENT_STATUS_UNKNOWN_PARAM, "unknown_param");
583         if (seen & (1U << index))
584             REJECT(AGENT_STATUS_DUPLICATE, "duplicate_param");
585         seen |= 1U << index;
586         if (param.type != rule_type(&tool_rules[index]))
587             REJECT(AGENT_STATUS_BAD_TYPE, "bad_param_type");
588         if (param.type == AGENT_PARAM_UINT64) {
589             if (param.value_size != sizeof(param.value.uint64_value))
590                 REJECT(AGENT_STATUS_BAD_SIZE, "bad_value_size");
591         } else if (param.value_size == 0 ||
592             param.value_size > AGENT_PARAM_STRING_SIZE ||
593             string_length(param.value.string_value, param.value_size,
594                 &text_length) < 0 ||
595             text_length + 1 != param.value_size)
596             REJECT(AGENT_STATUS_BAD_SIZE, "bad_value_size");
597         switch (rule_target(&tool_rules[index])) {
598             case PARAM_ARG0: op->arg0 = param.value.uint64_value; break;
599             case PARAM_ARG1: op->arg1 = param.value.uint64_value; break;
600             default: safestrcpy(op->payload, param.value.string_value,
601                 sizeof(op->payload)); break;
602         }
603     }
604     for (uint i = 0; i < count; i++)
605         if (rule_required(&tool_rules[i]) && !(seen & (1U << i)))
606             REJECT(AGENT_STATUS_BAD_PARAM, "missing_param");
607     return AGENT_STATUS_OK;

```

通过 schema 后，调用进入 lifecycle operation gate，再调用 agent_execute_one()。这个入口负责调用结果、输出 artifact 类型、决策原因、provenance label 和 Evidence ticket 的汇总；受 enforcement 的拒绝会尝试写入安全证据。若 critical record 的预留或提交失败，

direct/V3 拒绝会以 NO_SPACE 或 RETRY 返回，因而并不保证每个拒绝都有 ticket。

Code 7: V3 调用的解码、gate 与执行进入

c

```

2678     if ((req.param_count == 0 && req.params != 0) ||
2679         (req.param_count != 0 && req.params == 0)) {
2680         status = AGENT_STATUS_BAD_PARAM;
2681         safestrcpy(error, "bad_param_pointer", sizeof(error));
2682         goto rejected;
2683     }
2684     if (req.param_count != 0 &&
2685         user_range_check(p->pagetable, req.params,
2686             (uint64)req.param_count *
2687             sizeof(struct agent_param_v2), PTE_R) < 0) {
2688         status = AGENT_STATUS_DENIED;
2689         safestrcpy(error, "bad_param_range", sizeof(error));
2690         goto rejected;
2691     }
2692     memset(&decode_request, 0, sizeof(decode_request));
2693     decode_request.version = AGENT_CALL_VERSION_V2;
2694     decode_request.size = sizeof(decode_request);
2695     decode_request.tool_id = req.tool_id;
2696     decode_request.param_count = req.param_count;
2697     decode_request.request_id = req.request_id;
2698     decode_request.params = req.params;
2699     safestrcpy(decode_request.tool_name, req.tool_name,
2700         sizeof(decode_request.tool_name));
2701     status = agent_tool_protocol_decode_v2(
2702         p->pagetable, &decode_request, &tool, &op, error, sizeof(error));
2703     if (status != AGENT_STATUS_OK) {
2704         status = AGENT_STATUS_DENIED;
2705         denial_reason = AGENT_EXECUTION_REASON_CONTRACT_INVALID;
2706         goto rejected;
2707     }
2708     if (workflow_lifecycle_operation_enter(lifecycle) < 0) {
2709         status = AGENT_STATUS_RETRY;
2710         safestrcpy(error, "workflow_fenced", sizeof(error));
2711         goto out;
2712     }
2713     operation_entered = 1;
2714     if (agent_execute_one(p, &op, &result, agent_ticks(), &binding, 0, 0,
2715         &outcome) < 0)
2716         result.status = AGENT_STATUS_NO_SPACE;
2717     status = result.status;
2718     resp.sequence = result.sequence;
2719     resp.value0 = result.value0;
2720     resp.value1 = result.value1;
2721     resp.value2 = result.value2;
2722     resp.output_artifact_type = outcome.output_artifact_type;
2723     resp.decision_reason = outcome.decision_reason;
2724     resp.completion_flags = outcome.completion_flags;
2725     resp.output_provenance_labels = outcome.output_provenance_labels;
2726     resp.evidence_ticket = outcome.evidence_ticket;
2727     safestrcpy(resp.result, result.result, sizeof(resp.result));
2728     goto out;

```

Code 8 : 受控拒绝的安全证据与返回封装

c

```

2730 rejected:
2731     if (enforced && op.tool_id > 0) {
2732         if (op.request_id == 0) {
2733             /* Keep user correlation zero while giving Evidence a valid key. */
2734             op.request_id = (1ULL << 63) |
2735                 ((uint64)(uint)p->pid << 32) |
2736                 (uint)(p->agent_call_count + 1);
2737         }
2738         status = agent_tool_call_v3_security_denial(
2739             p, &binding, &op, status, denial_reason,
2740             &result, &outcome);
2741         resp.sequence = result.sequence;
2742         resp.output_artifact_type = outcome.output_artifact_type;
2743         resp.decision_reason = outcome.decision_reason;
2744         resp.output_provenance_labels =
2745             outcome.output_provenance_labels;
2746         resp.evidence_ticket = outcome.evidence_ticket;
2747         safestrcpy(resp.result, result.result, sizeof(resp.result));
2748         error[0] = 0;
2749     }
2750 out:
2751     if (operation_entered)
2752         workflow_lifecycle_operation_leave(lifecycle);
2753     if (enforced &&
2754         (status == AGENT_STATUS_DENIED || status == AGENT_STATUS_STALE) &&
2755         resp.evidence_ticket == 0) {
2756         status = AGENT_STATUS_RETRY;
2757         error[0] = 0;
2758         safestrcpy(resp.result, "security_evidence_unavailable",
2759             sizeof(resp.result));
2760     }
2761     resp.status = status;
2762     if (status != AGENT_STATUS_OK && error[0] != 0)
2763         safestrcpy(resp.result, error, sizeof(resp.result));
2764     return copyout(p->pagetable, respaddr, (char *)&resp, sizeof(resp));

```

合同 admission 将绑定的 lifecycle、contract generation、node id、deadline、工具号和资源 envelope 对齐到内核 record。Task Channel 的内部请求还被限制为“output none”的预检形式；被拒绝的 Task 不会因此发布一个伪造的 typed output。

Code 9 : 合同节点的 generation、资源与工具一致性检查

c

```

1157     if (!workflow_lifecycle_key_equal(binding->lifecycle, current)) {
1158         agent_execution_result_error(
1159             result, AGENT_STATUS_STALE, "stale_lifecycle",
1160             AGENT_EXECUTION_REASON_STALE_LIFECYCLE, claim);
1161         goto denied_counted;
1162     }
1163     if (!agent_execution_record_enforced(record, current)) {
1164         agent_execution_result_error(
1165             result, AGENT_STATUS_DENIED, "execution_contract_missing",
1166             AGENT_EXECUTION_REASON_CONTRACT_REQUIRED, claim);
1167         goto denied_counted;

```

```

1168     }
1169     if (record->state == AGENT_EXECUTION_CONTRACT_RETIRING) {
1170         agent_execution_result_error(
1171             result,
1172             (binding->internal_flags &
1173              AGENT_EXECUTION_BINDING_INTERNAL_F_TASK_CHANNEL) != 0 ?
1174             AGENT_STATUS_DENIED : AGENT_STATUS_CANCELLED,
1175             "contract_retiring",
1176             AGENT_EXECUTION_REASON_CONTRACT_RETIRING, claim);
1177         goto denied_counted;
1178     }
1179     if (binding->contract_generation == 0 ||
1180         binding->contract_generation != record->generation) {
1181         agent_execution_result_error(
1182             result, AGENT_STATUS_STALE, "stale_contract",
1183             AGENT_EXECUTION_REASON_STALE_CONTRACT, claim);
1184         goto denied_counted;
1185     }
1186     if (binding->node_id >= record->node_count) {
1187         agent_execution_result_error(
1188             result, AGENT_STATUS_DENIED, "unknown_contract_node",
1189             AGENT_EXECUTION_REASON_UNKNOWN_NODE, claim);
1190         goto denied_counted;
1191     }
1192     node = &record->nodes[binding->node_id];
1193     runtime = &record->runtime[binding->node_id];
1194     claim->output_artifact_type = node->output_artifact_type;
1195     claim->charge_class = node->charge_class;
1196     contract_deadline = node->deadline_tick != 0 &&
1197         (record->deadline_tick == 0 ||
1198          node->deadline_tick < record->deadline_tick) ?
1199         node->deadline_tick : record->deadline_tick;
1200     claim->deadline_tick = request_deadline_tick != 0 ?
1201         request_deadline_tick : contract_deadline;
1202     claim->manifest.accepted_input_labels = node->accepted_input_labels;
1203     claim->manifest.output_add_labels = node->output_add_labels;
1204     claim->manifest.required_capabilities = node->required_capabilities;
1205     claim->manifest.side_effect_mask = node->side_effect_mask;
1206     for (uint kind = 0; kind < RESOURCE_KIND_COUNT; kind++) {
1207         claim->exec_amounts[kind] = node->exec_envelope[kind];
1208         claim->storage_amounts[kind] = node->storage_envelope[kind];
1209     }
1210     if ((binding->internal_flags &
1211          AGENT_EXECUTION_BINDING_INTERNAL_F_TASK_CHANNEL) != 0) {
1212         uint expected_policy =
1213             AGENT_EXECUTION_PREFLIGHT_F_OUTPUT_NONE_ONLY |
1214             (request_deadline_tick != 0 ?
1215              AGENT_EXECUTION_PREFLIGHT_F_HARD_DEADLINE : 0);
1216
1217         if (admission_policy_flags != expected_policy ||
1218             (contract_deadline != 0 &&
1219              (admission_policy_flags &
1220               AGENT_EXECUTION_PREFLIGHT_F_HARD_DEADLINE) == 0) ||
1221             node->output_artifact_type != AGENT_ARTIFACT_NONE) {
1222             /* A rejected Task request never publishes a typed output. */
1223             claim->output_artifact_type = AGENT_ARTIFACT_NONE;
1224             agent_execution_result_error(

```

```

1225         result, AGENT_STATUS_DENIED,
1226         "task_admission_policy",
1227         AGENT_EXECUTION_REASON_CONTRACT_INVALID, claim);
1228         goto denied_counted;
1229     }
1230 } else if (admission_policy_flags != 0) {
1231     agent_execution_result_error(
1232         result, AGENT_STATUS_DENIED,
1233         "unexpected_admission_policy",
1234         AGENT_EXECUTION_REASON_CONTRACT_INVALID, claim);
1235     goto denied_counted;
1236 }
1237 if ((runtime->flags &
1238     AGENT_EXECUTION_RUNTIME_F_DEPENDENCY_TERMINAL) != 0) {
1239     claim->dependency_failed = 1;
1240     claim->retry_forbidden = 1;
1241     claim->decision_reason =
1242         AGENT_EXECUTION_REASON_DEPENDENCY_FAILED;
1243 }
1244 if (node->tool_id != (uint)op->tool_id) {
1245     agent_execution_result_error(
1246         result, AGENT_STATUS_DENIED, "contract_tool_mismatch",
1247         AGENT_EXECUTION_REASON_TOOL_MISMATCH, claim);
1248     goto denied_counted;
1249 }
1250 if (!agent_execution_digest_equal(binding->schema_digest,

```

效果开始前还要二次检查 node 仍在 running 状态、request/attempt/digest 与 claim 一致，并处理 timeout 或 cancel winner。该检查缩短了“合同通过”与“副作用实际发生”之间的窗口，保证状态过期时不继续产生内核可见效果。

Code 10 : 副作用开始前的合同 gate

c

```

1781 enum agent_execution_effect_admission
1782 agent_execution_contract_effect_begin(struct agent_execution_claim *claim)
1783 {
1784     struct agent_execution_contract_record *record;
1785     struct agent_execution_node_runtime *runtime;
1786     int enabled;
1787
1788     if (claim == 0 || !claim->active)
1789         return AGENT_EXECUTION_EFFECT_STALE;
1790     if (claim->legacy)
1791         return AGENT_EXECUTION_EFFECT_ALLOWED;
1792     if (claim->slot < 0 || claim->slot >= WORKFLOW_LIFECYCLE_CAP ||
1793         claim->node_id >= AGENT_EXECUTION_CONTRACT_MAX_NODES)
1794         return AGENT_EXECUTION_EFFECT_STALE;
1795     enabled = intr_save();
1796     record = &agent_execution_contracts[claim->slot];
1797     if (!agent_execution_record_matches(record, claim->lifecycle) ||
1798         record->generation != claim->contract_generation ||
1799         claim->node_id >= record->node_count) {
1800         intr_restore(enabled);
1801         return AGENT_EXECUTION_EFFECT_STALE;
1802     }

```

```

1803     runtime = &record->runtime[claim->node_id];
1804     if (runtime->state != AGENT_EXECUTION_NODE_RUNNING ||
1805         runtime->request_id != claim->request_id ||
1806         runtime->attempt_id != claim->attempt_id ||
1807         !agent_execution_digest_equal(runtime->request_digest,
1808                                       claim->request_digest)) {
1809         intr_restore(enabled);
1810         return AGENT_EXECUTION_EFFECT_STALE;
1811     }
1812     if ((runtime->flags &
1813         AGENT_EXECUTION_RUNTIME_F_TIMEOUT_REQUESTED) != 0) {
1814         claim->decision_reason =
1815             AGENT_EXECUTION_REASON_DEADLINE_EXPIRED;
1816         intr_restore(enabled);
1817         return AGENT_EXECUTION_EFFECT_STALE;
1818     }
1819     if ((runtime->flags &
1820         AGENT_EXECUTION_RUNTIME_F_CANCEL_REQUESTED) != 0) {
1821         claim->decision_reason =
1822             AGENT_EXECUTION_REASON_CANCEL_REQUESTED;
1823         if ((runtime->flags &
1824             AGENT_EXECUTION_RUNTIME_F_FORCE_CANCEL) != 0)
1825             claim->retry_forbidden = 1;
1826         intr_restore(enabled);
1827         return AGENT_EXECUTION_EFFECT_CANCELLED;
1828     }
1829     runtime->flags |= AGENT_EXECUTION_RUNTIME_F_EFFECT_STARTED;
1830     intr_restore(enabled);
1831     return AGENT_EXECUTION_EFFECT_ALLOWED;

```

Algorithm 3：一次 V3 工具调用的内核决策顺序

1. 验证固定大小 ABI、用户地址、Agent 身份与 workflow generation。
2. resolve 工具描述符并按 schema 解码参数；失败即构造结构化拒绝结果，critical record 无法建立时返回 NO_SPACE/RETRY。
3. admission 校验合同 DAG、输入 provenance、capability、deadline 和资源 envelope。
4. 开始 effect 前重新确认 claim；执行后发布 Context、Evidence、provenance 与 terminal result。

3.8.3. Context、回滚边界与 provenance

Context sidecar 只保存归一化的 agent_op/result 与归因，不完整保留 V2 parameter、V3 binding 或 Task SQE。mirror 的发布采用 prepare–begin–end 顺序；detail 在 record 标为 odd 前写入，sequence 只在一个 clear epoch 内单调，clear 后从 1 重新开始。Guest 看到的只读页因此可作为稳定快照使用，而不是观察到半写入的 record。

Code 11：Context sidecar 写入与发布窗口

c

```

1152 static int
1153 agent_context_append_flags(struct proc *p, struct agent_op *op,
1154                          struct agent_result *latest, uint64 tick,

```

```

1155         uint64 flags, int authority_effect,
1156         int causal_audit,
1157         struct agent_evidence_context_reservation *reservation,
1158         struct agent_evidence_security_reservation *
1159         security_reservation,
1160         uint64 *evidence_ticket_out)
1161 {
1162     struct agent_context_append_receipt receipt;
1163     struct agent_context_detail detail;
1164     struct agent_context_record record;
1165     struct thread *t = curr_thread();
1166     uint64 slot;
1167
1168     if (p == 0 || op == 0 || latest == 0 || p->agent_ctx_base == 0 ||
1169         p->context_path_capacity == 0 || !agent_context_layout_ok() ||
1170         p->agent_current_span_id == 0 ||
1171         p->agent_current_span_owner == 0 || t == 0 || t->process != p ||
1172         p->agent_context_lane_owner_tid != t->tid ||
1173         p->agent_context_lane_depth == 0 ||
1174         latest->sequence != p->agent_call_count)
1175         return -1;
1176     if ((reservation != 0 && security_reservation != 0) ||
1177         ((reservation != 0 || security_reservation != 0) &&
1178          (evidence_ticket_out == 0 ||
1179           agent_observe_recording_suppressed(p))) ||
1180         (reservation != 0 && !reservation->active) ||
1181         (security_reservation != 0 && !security_reservation->active))
1182         return -1;
1183     slot = p->context_path_head % p->context_path_capacity;
1184
1185     memset(&record, 0, sizeof(record));
1186     record.sequence = latest->sequence;
1187     record.request_id = latest->request_id;
1188     record.cause_sequence = p->agent_current_cause_sequence;
1189     record.span_id = p->agent_current_span_id;
1190     record.branch_generation = p->context_branch_generation;
1191     record.path_parent_sequence = p->context_path_visible_head;
1192     record.arg0 = op->arg0;
1193     record.value0 = latest->value0;
1194     record.value1 = latest->value1;
1195     record.value2 = latest->value2;
1196     record.tick = tick;
1197     record.flags = agent_provenance_context_flags(
1198         p, latest->request_id, latest->tool_id, flags);
1199     if (strlen(op->payload) >= sizeof(record.payload) ||
1200         strlen(latest->result) >= sizeof(record.result))
1201         record.flags |= AGENT_CONTEXT_RECORD_F_TRUNCATED;
1202     record.tool_id = latest->tool_id;
1203     record.status = latest->status;
1204     safestrcpy(record.payload, op->payload, sizeof(record.payload));
1205     safestrcpy(record.result, latest->result, sizeof(record.result));
1206     record.prev_hash = p->agent_context_chain_hash;
1207     record.record_hash = agent_context_record_hash(&record);
1208     if (agent_context_append_receipt_prepare(p, &record, slot, &receipt) < 0)
1209         return -1;
1210
1211     memset(&detail, 0, sizeof(detail));

```

```

1212 detail.sequence = latest->sequence;
1213 detail.flags = record.flags;
1214 memmove(&detail.op, op, sizeof(*op));
1215 memmove(&detail.result, latest, sizeof(*latest));
1216 if (agent_context_store(p, slot, &detail,
1217     p->agent_current_span_owner,
1218     p->agent_current_cause_pid,
1219     p->agent_current_cause_control) < 0)
1220     return -1;
1221 agent_context_publish_begin(p);
1222 agent_context_write_record(p, slot, &record);
1223 if (p->context_path_count < p->context_path_capacity) {
1224     if (p->context_path_count == 0)
1225         p->context_path_oldest = latest->sequence;
1226     p->context_path_count++;
1227 } else {
1228     p->context_path_oldest =
1229         latest->sequence - p->context_path_capacity + 1;
1230 }
1231 p->context_path_latest = latest->sequence;
1232 p->context_path_head = (slot + 1) % p->context_path_capacity;
1233 if (record.cause_sequence != 0)
1234     p->agent_provenance_edges++;
1235 if (agent_context_append_receipt_commit(p, &record, &receipt) < 0)
1236     panic("Agent context append receipt");
1237 p->agent_context_chain_hash = record.record_hash;
1238 p->agent_current_cause_sequence = latest->sequence;
1239 p->context_cause_branch_generation = record.branch_generation;
1240 p->agent_current_cause_pid = p->pid;
1241 p->agent_current_cause_control = p->agent_control_id;
1242 p->agent_current_span_id = record.span_id;
1243 agent_provenance_context_committed(
1244     p, record.request_id, record.tool_id, record.flags);
1245 if (agent_context_write_latest(p, latest) < 0 ||
1246     agent_context_write_header_locked(p) < 0)
1247     panic("Agent context direct publish");
1248 /* Reserved terminal and denial records stay behind an odd sequence until
1249  * their canonical Evidence ticket is committed. The release below is the
1250  * only point at which direct readers may accept the new Context snapshot. */
1251 if (reservation != 0)
1252     *evidence_ticket_out =
1253         agent_observe_commit_context_reserved_ticket(
1254             p, &record, authority_effect, causal_audit,
1255             reservation);
1256 else if (security_reservation != 0)
1257     *evidence_ticket_out =
1258         agent_observe_commit_security_reserved_ticket(
1259             p, &record, security_reservation);
1260 agent_context_publish_end(p);
1261 if (reservation != 0 || security_reservation != 0) {
1262     agent_observe_publish_context_ticket(
1263         p, &record, authority_effect, causal_audit,
1264         *evidence_ticket_out);

```

在执行入口中，Context append 准备发生在合同 admission 之前；输入 provenance label 随合同 claim 合并到当前进程。对于 deadline、依赖失败和策略拒绝，代码会尝试生成终态

记录或 security evidence，使调试和测试能区分“未执行”、“拒绝”与“执行后失败”；证据记录本身无法建立时，direct/V3 拒绝保留 NO_SPACE/RETRY 的可见失败语义。manual note、rollback 和 clear 则不走 Evidence 的预留/提交 ticket 协议。

Code 12 : 执行入口中的 Context 准备与 provenance 合并

c

```

1577 agent_execute_one(struct proc *p, struct agent_op *op,
1578                  struct agent_result *res, uint64 tick,
1579                  const struct agent_execution_binding *binding,
1580                  uint64 request_deadline_tick, uint admission_policy_flags,
1581                  struct agent_execution_outcome *outcome)
1582 {
1583     struct agent_execution_claim claim;
1584     struct agent_provenance_request provenance_request;
1585     struct agent_provenance_decision provenance_decision;
1586     struct agent_evidence_context_reservation evidence_reservation = { 0 };
1587     struct resource_phase_lease phase_lease = { 0 };
1588     struct resource_account_handle storage = resource_account_none();
1589     enum agent_execution_admission admission;
1590     enum agent_execution_effect_admission effect_admission;
1591     int bound = binding != 0;
1592     int phase_started = 0;
1593     uint64 phase_generation = 0;
1594     uint64 terminal_tick = tick;
1595     int result = 0;
1596     int status;
1597
1598     if (outcome != 0)
1599         memset(outcome, 0, sizeof(*outcome));
1600     op->payload[AGENT_OP_PAYLOAD_SIZE - 1] = 0;
1601     agent_result_init(res, op);
1602     if (agent_lifecycle_context_lane_enter(p) < 0) {
1603         res->status = AGENT_STATUS_NO_SPACE;
1604         agent_result_text(res, "context_interrupted");
1605         return -1;
1606     }
1607     if (agent_context_append_prepare(p, p->agent_call_count + 1) < 0) {
1608         res->status = AGENT_STATUS_NO_SPACE;
1609         agent_result_text(res, "context_sync_unavailable");
1610         agent_lifecycle_context_lane_leave(p);
1611         return -1;
1612     }
1613     admission = agent_execution_contract_admit(
1614         p, binding, op, request_deadline_tick, admission_policy_flags,
1615         tick, &claim, res, outcome);
1616     if (bound && admission == AGENT_EXECUTION_ADMISSION_EXECUTE &&
1617         claim.input_provenance_labels != 0 &&
1618         agent_provenance_merge_current(
1619             p, claim.input_provenance_labels) != AGENT_STATUS_OK)
1620         panic("contract input provenance merge");
1621     if (outcome != 0 && admission != AGENT_EXECUTION_ADMISSION_CACHED) {
1622         outcome->output_artifact_type = claim.output_artifact_type;
1623         outcome->terminal_tick = terminal_tick;
1624     }
1625     if (admission == AGENT_EXECUTION_ADMISSION_DENIED) {

```

```

1626     if (res->status == AGENT_STATUS_DENIED ||
1627         res->status == AGENT_STATUS_STALE) {
1628         result = agent_execution_security_denial(
1629             p, &claim, op, bound, res, outcome);
1630         agent_execution_publish_policy_denial(
1631             p, op, res, &claim);
1632         goto out;
1633     }
1634     p->agent_call_count++;
1635     res->sequence = p->agent_call_count;
1636     result = agent_context_append(p, op, res, tick, 0);
1637     if (result < 0)
1638         p->agent_call_count--;
1639     goto out;
1640 }
1641 if (admission == AGENT_EXECUTION_ADMISSION_CACHED) {
1642     if (outcome != 0)
1643         outcome->completion_flags |= AGENT_RESPONSE_V3_F_CACHED;
1644     goto out;
1645 }
1646 if (claim.deadline_expired || claim.dependency_failed) {
1647     if (agent_evidence_context_reserve(
1648         p, &evidence_reservation) < 0) {
1649         res->status = AGENT_STATUS_RETRY;
1650         agent_result_text(res, "evidence_reservation_unavailable");
1651         agent_execution_contract_abort(&claim);
1652         goto out;
1653     }
1654     p->agent_call_count++;
1655     res->sequence = p->agent_call_count;
1656     res->status = claim.deadline_expired ?
1657         AGENT_STATUS_TIMEOUT : AGENT_STATUS_CANCELLED;
1658     agent_result_text(
1659         res, claim.deadline_expired ?
1660             "contract_deadline" : "dependency_failed");
1661     if (outcome != 0) {
1662         outcome->decision_reason = claim.decision_reason;
1663         outcome->output_provenance_labels =
1664             agent_provenance_current_labels(p);
1665     }
1666     result = agent_execution_append_terminal(
1667         p, op, res, tick, 0, &claim, outcome,
1668         &evidence_reservation);

```

回滚接口恢复的是 Context Path 的可见记录与 branch 状态，不会撤销已经发生的文件写入、消息发送或外部 provider 调用。该边界在产品语义上被明确保留：Context rollback 适合修正后续推理与可见因果路径，不能被表述为通用事务回滚。

3.8.4. Metadata 查询与 Live Query 事件

文件查询先建立计划：显式 scan、关闭索引、或在启用索引时按固定 status->stage->kind 顺序取第一个已给定字段；它不比较基数。随后在 catalog read snapshot 内扫描候选记录，进行 scope visibility 与 predicate 匹配，并周期性让出 kernel work budget。结果同时返回 total hits/returned/truncated、used-index、reason、query ticks 与 fs generation，供用户态解释一

次查询的执行形态。

Code 13 : Metadata Query 的索引计划与快照执行

c

```

143 static void
144 agent_query_plan_build(const struct agent_file_query *q,
145                       struct agent_file_query_result *r,
146                       struct agent_query_plan *plan)
147 {
148     memset(plan, 0, sizeof(*plan));
149     plan->limit = q->max_hits;
150     if (plan->limit <= 0 || plan->limit > AGENT_FILE_QUERY_MAX_HITS)
151         plan->limit = AGENT_FILE_QUERY_MAX_HITS;
152     r->plan = AGENT_FILE_QUERY_PLAN_SCAN;
153     r->index_bucket = -1;
154     if (q->flags & AGENT_FILE_QUERY_SCAN) {
155         plan->force_scan = 1;
156         plan->reason |= AGENT_FILE_QUERY_REASON_FORCED_SCAN;
157         return;
158     }
159     if ((q->flags & AGENT_FILE_QUERY_USE_INDEX) == 0) {
160         plan->reason |= AGENT_FILE_QUERY_REASON_INDEX_OFF;
161         return;
162     }
163     for (int i = AGENT_QUERY_INDEX_FIRST; i < AGENT_QUERY_FIELDS; i++) {
164         const struct agent_query_alias *alias = &agent_query_aliases[i];
165         const char *value = (const char *)q + alias->offset;
166
167         if (!value[0])
168             continue;
169         plan->index = AGENT_CATALOG_INDEX_STATUS +
170             i - AGENT_QUERY_INDEX_FIRST;
171         plan->use_index = 1;
172         plan->index_key = value;
173         r->plan = AGENT_FILE_QUERY_PLAN_STATUS_INDEX +
174             i - AGENT_QUERY_INDEX_FIRST;
175         plan->reason |= AGENT_FILE_QUERY_REASON_STATUS_INDEX <<
176             (i - AGENT_QUERY_INDEX_FIRST);
177         return;
178     }
179     plan->reason |= AGENT_FILE_QUERY_REASON_NO_INDEX_KEY;
180 }
181
182 static void agent_query_result_reset(struct agent_file_query_result *r)
183 {
184     memset(r, 0, sizeof(*r));
185 }
186
187 int
188 agent_metadata_query_execute_snapshot(
189     uint scope, const struct agent_file_query *q,
190     struct agent_file_query_result *r)
191 {
192     struct agent_catalog_read_snapshot snapshot;
193     struct agent_file_meta meta;
194     struct agent_query_plan plan;

```

```

195     uint owner;
196     uint work = 0;
197     int bucket = -1;
198     int copied;
199     uint64 start;
200
201     agent_query_result_reset(r);
202     agent_query_plan_build(q, r, &plan);
203     start = agent_file_state_now();
204     copied = agent_metadata_catalog_read_begin(
205         scope, plan.index, plan.index_key, plan.force_scan,
206         &snapshot, &bucket);
207     if (copied < 0)
208         return copied;
209     if (plan.use_index)
210         r->index_bucket = bucket;
211     for (int slot = agent_metadata_catalog_read_next(&snapshot, -1);
212          slot >= 0;
213          slot = agent_metadata_catalog_read_next(&snapshot, slot)) {
214         r->scanned_records++;
215         copied = agent_metadata_catalog_read_copy(
216             &snapshot, slot, &meta, &owner);
217         if (copied < 0)
218             return copied;
219         if (copied > 0 && agent_object_scope_visible(scope, owner)) {
220             r->candidate_records++;
221             if (agent_metadata_query_matches(scope, owner, q, &meta)) {
222                 r->total_hits++;
223                 if (r->returned < plan.limit) {
224                     agent_file_state_project_hit(
225                         &r->hits[r->returned++],
226                         &meta, owner);
227                 } else {
228                     r->truncated = 1;
229                 }
230             }
231         }
232         if (++work == AGENT_QUERY_READ_GRANULE) {
233             work = 0;
234             (void)kernel_work_checkpoint_cleanup(
235                 KERNEL_WORK_OPERATION_UNITS);
236         }
237     }
238     if (!agent_metadata_catalog_read_end(&snapshot))
239         return AGENT_CATALOG_STALE;
240     r->used_index = plan.use_index;
241     r->index_rebuild_records = 0;
242     r->query_ticks = agent_file_state_now() - start;
243     r->plan_reason = plan.reason;
244     r->fs_generation = snapshot.fs_generation;
245     return r->returned;

```

Live Query 事件把 metadata 的前后状态投影为 ENTER、UPDATE、LEAVE 三类变化。subscription 保存 predicate 与世代，而不保存结果集；每个目标先经过 workflow/scope 可见性检查。只有投递失败或队列缺口才置 resync，正常 fs generation 变化继续投递三类增量。

安全恢复保持旧 watch，先装同 query、无 ACK_RESYNC 的 replacement，取得 truncated=0 snapshot 后才在旧 watch ACK/unwatch；replacement 覆盖切换窗口。Fence 排空延迟工作；workflow/scope domain 的 sticky resync 未 ACK 时，即使 marker 已投递仍返回 RETRY，最终 reclaim 清理残留。

Code 14 : Live Query 的 ENTER、UPDATE、LEAVE 事件投递

c

```

1250     struct workflow_lifecycle_key source_key =
1251         agent_live_query_proc_key(source);
1252     uint source_scope = agent_identity_proc_scope(source);
1253     uint64 fid;
1254     int delivered = 0;
1255
1256     if (!agent_live_query_domain_valid(key, owner_scope) ||
1257         identity == 0 || !identity->used || identity->fid <= 0)
1258         return 0;
1259     if (generation == 0)
1260         generation = agent_file_state_scope_generation(owner_scope);
1261     fid = (uint64)(uint)identity->fid;
1262     for (struct proc *target = pool; target < &pool[NPROC]; target++) {
1263         struct workflow_lifecycle_key target_key;
1264         struct proc *attributed = 0;
1265         uint target_scope;
1266         uint legacy_changes;
1267         uint typed_changes;
1268         int enabled = intr_save();
1269
1270         if (!agent_live_query_target_visible(target, key, owner_scope) ||
1271             !agent_live_query_target_valid(
1272                 target, &target_key, &target_scope)) {
1273             intr_restore(enabled);
1274             continue;
1275         }
1276         legacy_changes = agent_live_query_target_changes(
1277             target, owner_scope, before, after);
1278         typed_changes = agent_live_query_typed_target_changes(
1279             target, owner_scope, before, after);
1280         if (legacy_changes == 0 && typed_changes == 0) {
1281             intr_restore(enabled);
1282             continue;
1283         }
1284         if (source != 0 && source_scope == target_scope &&
1285             agent_live_query_key_equal(source_key, target_key))
1286             attributed = source;
1287         intr_restore(enabled);
1288         for (uint stream = 0; stream < 2; stream++) {
1289             uint changes = stream == 0 ? legacy_changes : typed_changes;
1290             int event_type = stream == 0 ? AGENT_EVENT_FILE_STATUS :
1291                 AGENT_EVENT_FILE_QUERY;
1292
1293             for (int change = AGENT_LIVE_QUERY_ENTER;
1294                 change <= AGENT_LIVE_QUERY_LEAVE; change++) {
1295                 char payload[AGENT_EVENT_PAYLOAD_SIZE];
1296                 int result;
1297

```

```

1298         if ((changes & (1U << change)) == 0)
1299             continue;
1300         agent_live_query_payload(payload, change, key);
1301         result = agent_ipc_deliver_live_event(
1302             target, attributed, event_type, fid,
1303             generation, payload, 0);
1304         if (result > 0) {
1305             delivered += result;
1306             continue;
1307         }
1308         if (result < 0) {
1309             enabled = intr_save();
1310             agent_live_query_proc_resync_mark_locked(
1311                 target, generation,
1312                 1U << event_type);
1313             intr_restore(enabled);
1314             (void)agent_live_query_proc_resync_flush(
1315                 target);
1316         }
1317     }
1318 }
1319 }
1320 return delivered;
1321 }

```

Code 15 : Live Query 的 resync ACK 与 Fence 入口

c

```

1323 void
1324 agent_live_query_watch_installed(struct proc *p)
1325 {
1326     struct workflow_lifecycle_key key;
1327     uint64 generation;
1328     uint scope_id;
1329     int enabled = intr_save();
1330
1331     if (!agent_live_query_target_valid(p, &key, &scope_id) ||
1332         !agent_live_query_has_file_watch(p))
1333         goto out;
1334     generation = agent_file_state_scope_generation(scope_id);
1335     generation = MAX(generation,
1336         agent_live_query_domain_generation_locked(key, scope_id));
1337     agent_live_query_proc_resync_mark_locked(
1338         p, generation, 1U << AGENT_EVENT_FILE_STATUS);
1339     (void)agent_live_query_proc_resync_flush(p);
1340 out:
1341     intr_restore(enabled);
1342 }
1343
1344 void
1345 agent_live_query_watch_removed(struct proc *p)
1346 {
1347     int enabled;
1348     uint slot;
1349
1350     if (p == 0 || p < pool || p >= &pool[NPROC])
1351         return;

```

```

1352     enabled = intr_save();
1353     if (!agent_live_query_has_file_watch(p)) {
1354         slot = p - pool;
1355         agent_live_query_proc_resync_clear(
1356             &agent_live_query_proc_resync[slot]);
1357     }
1358     intr_restore(enabled);
1359 }
1360
1361 void
1362 agent_live_query_proc_reset(struct proc *p)
1363 {
1364     int enabled;
1365
1366     if (p == 0 || p < pool || p >= &pool[NPROC])
1367         return;
1368     enabled = intr_save();
1369     for (uint slot = 0; slot < AGENT_LIVE_QUERY_SUBSCRIPTION_CAP; slot++)
1370         if (agent_live_query_subscriptions[slot].used &&
1371             agent_live_query_subscriptions[slot].target == p)
1372             agent_live_query_subscription_remove_locked(
1373                 &agent_live_query_subscriptions[slot], 0);
1374     agent_live_query_proc_resync_clear(
1375         &agent_live_query_proc_resync[p - pool]);
1376     intr_restore(enabled);
1377 }
1378
1379 int
1380 agent_live_query_resync_ack(struct workflow_lifecycle_key key,
1381     uint scope_id, uint64 generation)
1382 {
1383     int acknowledged = 0;
1384     int enabled;
1385
1386     if (!agent_live_query_domain_valid(key, scope_id) || generation == 0)
1387         return 0;
1388     enabled = intr_save();
1389     for (uint slot = 0; slot < AGENT_LIVE_QUERY_DOMAIN_CAP; slot++) {
1390         struct agent_live_query_domain_resync *state =
1391             &agent_live_query_domain_resync[slot];
1392
1393         if (!state->used || state->generation > generation ||
1394             !agent_live_query_domain_equal(
1395                 state->key, state->scope_id, key, scope_id))
1396             continue;
1397         memset(state, 0, sizeof(*state));
1398         acknowledged = 1;
1399     }
1400     if (scope_id == VFS_SCOPE_SYSTEM &&
1401         agent_live_query_global_resync_generation != 0 &&
1402         agent_live_query_global_resync_generation <= generation) {
1403         agent_live_query_global_resync_generation = 0;
1404         acknowledged = 1;
1405     }
1406     intr_restore(enabled);
1407     return acknowledged;
1408 }

```

```

1409
1410 static int
1411 agent_live_query_proc_resync_pending_domain(
1412     struct workflow_lifecycle_key key, uint scope_id)
1413 {
1414     for (struct proc *target = pool; target < &pool[NPROC]; target++) {
1415         struct agent_live_query_proc_resync *pending =
1416             &agent_live_query_proc_resync[target - pool];
1417
1418         if (!agent_live_query_proc_resync_valid(pending, target))
1419             continue;
1420         if (scope_id == VFS_SCOPE_SYSTEM ||
1421             agent_live_query_domain_equal(
1422                 pending->key, pending->scope_id, key, scope_id))
1423             return 1;
1424     }
1425     return 0;
1426 }
1427
1428 int
1429 agent_live_query_fence_drain(struct workflow_lifecycle_key key, uint scope_id)
1430 {
1431     int retry = 0;
1432
1433     if (!agent_live_query_domain_valid(key, scope_id) ||
1434         !agent_metadata_txn_owned(0))
1435         return AGENT_STATUS_RETRY;
1436     for (uint slot = 0; slot < AGENT_LIVE_QUERY_TOMBSTONE_CAP; slot++) {
1437         int pending;
1438         int enabled = intr_save();
1439
1440         pending = agent_live_query_tombstones[slot].used &&
1441             agent_live_query_domain_equal(
1442                 agent_live_query_tombstones[slot].key,
1443                 agent_live_query_tombstones[slot].scope_id,
1444                 key, scope_id);
1445         intr_restore(enabled);
1446         if (!pending)
1447             continue;
1448         if (agent_live_query_tombstone_process(slot) < 0)
1449             retry = 1;
1450     }
1451     for (uint slot = 0; slot < AGENT_FILE_META_MAX; slot++) {
1452         int pending;
1453         int enabled = intr_save();
1454
1455         pending = agent_live_query_content_pending[slot].used &&
1456             agent_live_query_domain_equal(
1457                 agent_live_query_content_pending[slot]
1458                     .receipt.lifecycle,
1459                 agent_live_query_content_pending[slot]
1460                     .receipt.scope_id,
1461                 key, scope_id);
1462         intr_restore(enabled);
1463         if (!pending)
1464             continue;
1465         if (agent_live_query_content_process(slot) < 0)

```

```

1466         retry = 1;
1467     }
1468     for (struct proc *target = pool; target < &pool[NPROC]; target++) {

```

3.8.5. Task Channel 与等待路径

Task setup 仅允许 Agent 主线程；SQ 和 CQ 分别占一个 Guest 映射页，复制的请求/终态和权威资源状态分别位于两个内核私有页。enter 与 resource binding 绑定 issuer tid 和 identity generation。进入内核时仍检查 enter ABI、CQ acknowledgement、resync 标志和 SQ 边界；issuer、owner 或 lifecycle 的 STALE 仅保留 ABI version/size 与 status，其余输出为零，control generation mismatch 则由 enter result fill 返回当前 channel state。CQ_FULL 只是 completion backpressure，不会把 accepted request 直接变成 terminal；target 已 ACK 或不存在的 cancel 仍消费 cancel id，enter result fill 返回当前 generation 与 waterlines，且不额外生成 cancel CQE。ring/slot 协议故障进入 sticky resync 后，issuer 必须以空 SQ tail 的显式 resync enter 复位。这里的“queue”是受 contract 和证据约束的内核对象，不是普通用户态环形数组。

Code 16 : Task Channel enter 的 generation、resync 与 SQ 边界检查

c

```

1937 int
1938 agent_task_channel_enter(struct proc *p, struct thread *issuer,
1939                         const struct agent_task_channel_enter *enter,
1940                         struct agent_task_channel_enter_result *result,
1941                         const struct agent_task_channel_ops *ops)
1942 {
1943     struct agent_task_channel_state *state;
1944     struct agent_task_private_header *header;
1945     uint submitted = 0;
1946     uint completed = 0;
1947     uint limit;
1948     int status = AGENT_TASK_CHANNEL_OK;
1949     int enabled;
1950
1951     if (result == 0)
1952         return AGENT_TASK_CHANNEL_BAD_REQUEST;
1953     memset(result, 0, sizeof(*result));
1954     result->version = AGENT_TASK_CHANNEL_VERSION;
1955     result->size = sizeof(*result);
1956     result->status = AGENT_TASK_CHANNEL_BAD_REQUEST;
1957     if (p == 0 || issuer == 0 || enter == 0 ||
1958         enter->version != AGENT_TASK_CHANNEL_VERSION ||
1959         enter->size != sizeof(*enter) ||
1960         (enter->flags & ~AGENT_TASK_CHANNEL_ENTER_F_ALL) != 0 ||
1961         ((enter->flags & AGENT_TASK_CHANNEL_ENTER_F_DRAIN) != 0 &&
1962          enter->max_submit != 0) ||
1963         enter->max_submit > AGENT_TASK_CHANNEL_CAPACITY ||
1964         enter->min_complete > AGENT_TASK_CHANNEL_CAPACITY ||
1965         enter->reserved != 0 || enter->reserved_tail[0] != 0 ||
1966         enter->reserved_tail[1] != 0)
1967         return result->status;
1968     limit = (enter->flags & AGENT_TASK_CHANNEL_ENTER_F_DRAIN) != 0 ? 0 :
1969             (enter->max_submit == 0 ? AGENT_TASK_CHANNEL_CAPACITY :
1970              enter->max_submit);

```

```

1971     enabled = intr_save();
1972     state = agent_task_channel_find_locked(p);
1973     if (!agent_task_channel_issuer_valid_locked(state, p, issuer) ||
1974         state->ops != ops) {
1975         intr_restore(enabled);
1976         result->status = AGENT_TASK_CHANNEL_STALE;
1977         return result->status;
1978     }
1979     header = &state->private->header;
1980     if (enter->generation != header->generation) {
1981         agent_task_channel_enter_result_fill(
1982             state, result, AGENT_TASK_CHANNEL_STALE, 0, 0);
1983         intr_restore(enabled);
1984         return result->status;
1985     }
1986     status = agent_task_channel_ack_locked(state, enter->cq_head);
1987     if (status < 0) {
1988         agent_task_channel_enter_result_fill(state, result, status, 0, 0);
1989         intr_restore(enabled);
1990         return result->status;
1991     }
1992     header = &state->private->header;
1993     if ((header->flags & AGENT_TASK_CHANNEL_RING_F_RESYNC) != 0) {
1994         if ((enter->flags & AGENT_TASK_CHANNEL_ENTER_F_RESYNC) == 0 ||
1995             enter->sq_tail != 0) {
1996             agent_task_channel_enter_result_fill(
1997                 state, result,
1998                 AGENT_TASK_CHANNEL_RESYNC_REQUIRED, 0, 0);
1999             intr_restore(enabled);
2000             return result->status;
2001         }
2002         header->flags &= ~AGENT_TASK_CHANNEL_RING_F_RESYNC;
2003     } else if ((enter->flags & AGENT_TASK_CHANNEL_ENTER_F_RESYNC) != 0) {
2004         agent_task_channel_enter_result_fill(
2005             state, result, AGENT_TASK_CHANNEL_BAD_REQUEST, 0, 0);
2006         intr_restore(enabled);
2007         return result->status;
2008     }
2009     if (enter->sq_tail < header->sq_tail ||
2010         enter->sq_tail < header->sq_head ||
2011         enter->sq_tail - header->sq_head > AGENT_TASK_CHANNEL_CAPACITY) {
2012         status = agent_task_channel_protocol_fault_locked(state);
2013         agent_task_channel_enter_result_fill(state, result, status, 0, 0);
2014         intr_restore(enabled);
2015         return result->status;
2016     }
2017     header->sq_tail = enter->sq_tail;
2018     completed += agent_task_channel_flush_locked(state);
2019     agent_task_channel_publish_locked(state);
2020     intr_restore(enabled);

```

Task 的资源导入接口被设计为拒绝路径，不能把任意已占用资源转入任务作为隐式预算。这一限制使 admission 能继续依据合同声明的 envelope 作出可预测决定，代价是复杂的资源移交需要经显式的 workflow 资源接口完成。

3.8.6. Phase Lease 与 Workflow EEVDF

资源控制器以 phase lease 锁定并激活一组 account 预算。lease 记录 owner、lifecycle、generation 与账户对；激活时把 lease token 写入 thread cold state，deactivate/settle/abort 时必须匹配同一 generation。这给一段工具执行提供了受限、可结算的资源阶段。

Code 17 : Phase Lease 的创建与账户对提交

c

```

1770 int resource_phase_lease_begin(
1771     struct resource_phase_lease *lease,
1772     struct workflow_lifecycle_key lifecycle,
1773     struct resource_account_handle exec_handle,
1774     struct resource_account_handle storage_handle,
1775     enum resource_charge_class charge_class,
1776     const uint64 exec_amounts[RESOURCE_KIND_COUNT],
1777     const uint64 storage_amounts[RESOURCE_KIND_COUNT], uint node_id,
1778     uint64 request_id)
1779 {
1780     struct resource_account *accounts[RESOURCE_PHASE_ACCOUNT_COUNT];
1781     struct resource_phase_account_lock
1782         *locks[RESOURCE_PHASE_ACCOUNT_COUNT];
1783     struct resource_account_handle handles[RESOURCE_PHASE_ACCOUNT_COUNT] = {
1784         [RESOURCE_PHASE_EXEC] = exec_handle,
1785         [RESOURCE_PHASE_STORAGE] = storage_handle,
1786     };
1787     uint64 amounts[RESOURCE_PHASE_ACCOUNT_COUNT][RESOURCE_KIND_COUNT];
1788     uint masks[RESOURCE_PHASE_ACCOUNT_COUNT], union_mask;
1789     struct resource_phase_registry_entry *entry;
1790     struct thread *owner;
1791     int registry_slot;
1792     int enabled;
1793
1794     if (lease == 0 || lease->state != RESOURCE_PHASE_LEASE_EMPTY ||
1795         exec_amounts == 0 || storage_amounts == 0 || request_id == 0 ||
1796         !workflow_lifecycle_key_valid(lifecycle) ||
1797         !resource_charge_class_valid(charge_class) ||
1798         resource_account_handle_equal(exec_handle, storage_handle))
1799         return -1;
1800     memmove(amounts[RESOURCE_PHASE_EXEC], exec_amounts,
1801             sizeof(amounts[RESOURCE_PHASE_EXEC]));
1802     memmove(amounts[RESOURCE_PHASE_STORAGE], storage_amounts,
1803             sizeof(amounts[RESOURCE_PHASE_STORAGE]));
1804     for (uint role = 0; role < RESOURCE_PHASE_ACCOUNT_COUNT; role++) {
1805         for (uint kind = 0; kind < RESOURCE_KIND_COUNT; kind++)
1806             if (amounts[role][kind] > (uint)-1)
1807                 return -1;
1808         masks[role] = resource_phase_amount_mask(amounts[role]);
1809     }
1810     union_mask = masks[RESOURCE_PHASE_EXEC] |
1811                 masks[RESOURCE_PHASE_STORAGE];
1812     if (union_mask == 0)
1813         return -1;
1814
1815     enabled = intr_save();
1816     owner = curr_thread();

```

```

1817 registry_slot = resource_phase_registry_free_slot();
1818 accounts[RESOURCE_PHASE_EXEC] = resource_account_lookup(exec_handle);
1819 accounts[RESOURCE_PHASE_STORAGE] =
1820     resource_account_lookup(storage_handle);
1821 if (accounts[RESOURCE_PHASE_EXEC] == 0 ||
1822     accounts[RESOURCE_PHASE_STORAGE] == 0 ||
1823     accounts[RESOURCE_PHASE_EXEC]->kind != RESOURCE_ACCOUNT_EXEC ||
1824     accounts[RESOURCE_PHASE_STORAGE]->kind !=
1825         RESOURCE_ACCOUNT_STORAGE ||
1826     registry_slot < 0 || owner == 0 || owner->trapframe == 0 ||
1827     owner->identity_generation == 0 || owner->process == 0 ||
1828     !proc_tear_down_live(owner->process) ||
1829     resource_phase_registry_owner_busy(owner) ||
1830     charge_class !=
1831         (owner->process->resource_slot_reserved ?
1832             RESOURCE_CHARGE_RESERVED :
1833             RESOURCE_CHARGE_ORDINARY) ||
1834     resource_phase_generation == RESOURCE_LIMIT_UNBOUNDED ||
1835     !workflow_lifecycle_active(lifecycle) ||
1836     !resource_phase_owner_binding_matches(
1837         lifecycle, exec_handle, owner) ||
1838     !resource_phase_lifecycle_pair_admissible(
1839         lifecycle, handles) ||
1840     !resource_phase_lock_records_locked(accounts, handles, locks) ||
1841     !resource_phase_pair_admissible_locked(
1842         accounts, charge_class, amounts, masks, union_mask)) {
1843     intr_restore(enabled);
1844     return -1;
1845 }
1846 resource_phase_pair_reclaim_locked(
1847     accounts, charge_class, amounts, union_mask);
1848 resource_phase_pair_commit_locked(
1849     accounts, handles, locks, charge_class, amounts, masks);
1850 resource_phase_generation++;
1851 entry = &resource_phase_registry[registry_slot];
1852 memset(entry, 0, sizeof(*entry));
1853 entry->owner = owner;
1854 entry->lease.lifecycle = lifecycle;
1855 entry->lease.generation = resource_phase_generation;
1856 entry->lease.request_id = request_id;
1857 entry->lease.owner_thread_generation = owner->identity_generation;
1858 entry->lease.node_id = node_id;
1859 entry->lease.charge_class = charge_class;
1860 entry->lease.state = RESOURCE_PHASE_LEASE_ADMITTED;
1861 for (uint role = 0; role < RESOURCE_PHASE_ACCOUNT_COUNT; role++) {
1862     entry->lease.account[role].handle = handles[role];
1863     entry->lease.account[role].kind_mask = masks[role];
1864     for (uint kind = 0; kind < RESOURCE_KIND_COUNT; kind++)
1865         entry->lease.account[role].remaining[kind] =
1866             (uint) amounts[role][kind];
1867 }
1868 entry->used = 1;
1869 lease->generation = resource_phase_generation;
1870 lease->request_id = request_id;
1871 lease->node_id = node_id;
1872 lease->registry_slot = (uint) registry_slot;
1873 lease->state = RESOURCE_PHASE_LEASE_ADMITTED;

```

```

1874     intr_restore(enabled);
1875     return 0;
1876 }

```

Code 18 : Phase Lease 的线程激活与去激活

c

```

1878 int resource_phase_lease_activate(struct resource_phase_lease *lease,
1879                                  uint64 expected_generation)
1880 {
1881     struct thread *thread;
1882     struct resource_account *account;
1883     struct resource_phase_registry_entry *entry;
1884     struct resource_phase_lease_record *record;
1885     int enabled;
1886
1887     if (lease == 0 || expected_generation == 0)
1888         return -1;
1889     enabled = intr_save();
1890     thread = curr_thread();
1891     entry = resource_phase_registry_lookup(
1892         lease, expected_generation);
1893     record = entry == 0 ? 0 : &entry->lease;
1894     if (entry == 0 || record->state != RESOURCE_PHASE_LEASE_ADMITTED ||
1895         thread == 0 || entry->owner != thread ||
1896         record->owner_thread_generation != thread->identity_generation ||
1897         thread->trapframe == 0 ||
1898         thread->identity_generation == 0 ||
1899         thread_trap_cold(thread)->resource_phase_lease_generation != 0 ||
1900         thread_trap_cold(thread)->resource_phase_claim_depth != 0 ||
1901         !proc_tearardown_live(thread->process) ||
1902         !workflow_lifecycle_active(record->lifecycle) ||
1903         !resource_phase_owner_matches(record, thread))
1904         goto fail;
1905     for (uint role = 0; role < RESOURCE_PHASE_ACCOUNT_COUNT; role++) {
1906         account = resource_account_lookup(record->account[role].handle);
1907         if (account == 0 || account->state != RESOURCE_ACCOUNT_ACTIVE ||
1908             resource_phase_account_lock_lookup(
1909                 record->account[role].handle) == 0)
1910             goto fail;
1911     }
1912     record->owner_thread_generation = thread->identity_generation;
1913     record->state = RESOURCE_PHASE_LEASE_ACTIVE;
1914     lease->state = RESOURCE_PHASE_LEASE_ACTIVE;
1915     thread_trap_cold(thread)->resource_phase_lease_slot =
1916         lease->registry_slot;
1917     thread_trap_cold(thread)->resource_phase_lease_generation =
1918         expected_generation;
1919     intr_restore(enabled);
1920     return 0;
1921 fail:
1922     intr_restore(enabled);
1923     return -1;
1924 }
1925
1926 int resource_phase_lease_deactivate(struct resource_phase_lease *lease,

```

```

1927         uint64 expected_generation)
1928 {
1929     struct thread *thread;
1930     struct resource_phase_registry_entry *entry;
1931     struct resource_phase_lease_record *record;
1932     int enabled;
1933
1934     if (lease == 0 || expected_generation == 0)
1935         return -1;
1936     enabled = intr_save();
1937     thread = curr_thread();
1938     entry = resource_phase_registry_lookup(
1939         lease, expected_generation);
1940     record = entry == 0 ? 0 : &entry->lease;
1941     if (entry == 0 || record->state != RESOURCE_PHASE_LEASE_ACTIVE ||
1942         thread == 0 || entry->owner != thread ||
1943         thread->trapframe == 0 ||
1944         thread_trap_cold(thread)->resource_phase_lease_slot !=
1945             lease->registry_slot ||
1946         thread_trap_cold(thread)->resource_phase_lease_generation !=
1947             expected_generation ||
1948         record->owner_thread_generation != thread->identity_generation ||
1949         record->outstanding_claims != 0 ||
1950         thread_trap_cold(thread)->resource_phase_claim_depth != 0) {
1951         intr_restore(enabled);
1952         return -1;
1953     }
1954     thread_trap_cold(thread)->resource_phase_lease_slot =
1955         RESOURCE_PHASE_REGISTRY_SLOT_NONE;
1956     thread_trap_cold(thread)->resource_phase_lease_generation = 0;
1957     record->state = RESOURCE_PHASE_LEASE_DEACTIVATED;
1958     lease->state = RESOURCE_PHASE_LEASE_DEACTIVATED;
1959     intr_restore(enabled);
1960     return 0;
1961 }

```

调度器不把同一 workflow 中更多线程当作更多独立公平实体。workflow_scheduler_select() 先收集 runnable 工作流，再根据 lag、deadline、weight 和可运行负载选择实体；dispatch 与 charge 分别记录服务开始和已消耗时间。调度公平性的记账主体因而是 workflow，而非偶然的线程数量。

Code 19 : Workflow EEVDF 的候选收集与选择

c

```

408 int workflow_scheduler_select(
409     const struct workflow_scheduler_candidate *candidates, uint count,
410     uint64 now_cycle, uint64 now_tick, int *selected_domain)
411 {
412     int slots[WORKFLOW_SCHEDULER_ENTITY_CAP];
413     uint planned_latency[WORKFLOW_SCHEDULER_ENTITY_CAP];
414     uint planned_request_ticks[WORKFLOW_SCHEDULER_ENTITY_CAP];
415     uint planned_decays[WORKFLOW_SCHEDULER_ENTITY_CAP];
416     uint64 planned_vruntime[WORKFLOW_SCHEDULER_ENTITY_CAP];
417     uint64 planned_remaining[WORKFLOW_SCHEDULER_ENTITY_CAP];
418     uint64 planned_deadline[WORKFLOW_SCHEDULER_ENTITY_CAP];
419     uint reserved = 0;

```

```

420     uint new_slots = 0;
421     uint wake_transitions = 0;
422     uint64 rebase_offset;
423     uint64 planned_vtime;
424     int selected = -1;
425     int enabled;
426
427     (void)now_cycle;
428     if (selected_domain == 0 || candidates == 0 || count == 0 ||
429         count > WORKFLOW_SCHEDULER_ENTITY_CAP)
430         return -1;
431     *selected_domain = -1;
432     for (uint i = 0; i < WORKFLOW_SCHEDULER_ENTITY_CAP; i++)
433         slots[i] = WORKFLOW_SCHEDULER_SLOT_NONE;
434     enabled = intr_save();
435     /* Phase one is read-only: validate every identity and reserve local slots. */
436     for (uint i = 0; i < count; i++) {
437         int is_new = 0;
438
439         if (!workflow_scheduler_identity_valid(
440             candidates[i].lifecycle, candidates[i].account,
441             candidates[i].domain_id) ||
442             candidates[i].runnable_threads == 0 ||
443             candidates[i].runnable_threads > 0xffffU ||
444             candidates[i].latency_class >
445                 AGENT_WORKFLOW_LATENCY_BATCH)
446             goto fail;
447         for (uint j = 0; j < i; j++)
448             if (candidates[j].domain_id ==
449                 candidates[i].domain_id ||
450                 workflow_lifecycle_key_equal(
451                     candidates[j].lifecycle,
452                     candidates[i].lifecycle) ||
453                 resource_account_handle_equal(
454                     candidates[j].account,
455                     candidates[i].account))
456                 goto fail;
457         planned_request_ticks[i] = workflow_scheduler_request_ticks(
458             candidates[i].latency_class,
459             candidates[i].deadline_delta_ticks);
460         if (planned_request_ticks[i] == 0)
461             goto fail;
462         slots[i] = workflow_scheduler_plan_slot_locked(
463             &candidates[i], reserved, &is_new);
464         if (slots[i] < 0)
465             goto fail;
466         reserved |= 1U << slots[i];
467         if (is_new)
468             new_slots |= 1U << i;
469         else if (workflow_scheduler_entities[slots[i]].flags &
470                 WORKFLOW_SCHEDULER_F_CACHE_INVALID)
471             goto fail;
472     }
473
474     /* Compute the entire post-selection state locally before any mutation. */
475     rebase_offset = workflow_scheduler_rebase_offset_locked();
476     planned_vtime = workflow_scheduler_vtime >= rebase_offset ?

```

```

477         workflow_scheduler_vtime - rebase_offset : 0;
478     for (uint i = 0; i < count; i++) {
479         struct workflow_scheduler_entity *entity =
480             &workflow_scheduler_entities[slots[i]];
481         uint64 request_cycles = workflow_scheduler_request_cycles(
482             planned_request_ticks[i]);
483
484         planned_decays[i] = 0;
485         if (new_slots & (1U << i)) {
486             planned_vruntime[i] = planned_vtime;
487             planned_remaining[i] = 0;
488             planned_latency[i] = AGENT_WORKFLOW_LATENCY_NORMAL;
489             wake_transitions |= 1U << i;
490         } else {
491             planned_vruntime[i] = entity->vruntime - rebase_offset;
492             planned_remaining[i] = entity->remaining_cycles;
493             planned_latency[i] = entity->latency_class;
494             planned_request_ticks[i] = entity->request_ticks;
495             if (!(entity->flags &
496                 AGENT_WORKFLOW_SCHED_F_RUNNABLE)) {
497                 wake_transitions |= 1U << i;
498                 planned_remaining[i] = 0;
499                 if (entity->flags &
500                     AGENT_WORKFLOW_SCHED_F_SLEEPING) {
501                     uint64 slept =
502                         now_tick >= entity->sleep_start_tick ?
503                         now_tick -
504                         entity->sleep_start_tick :
505                         0;
506
507                     planned_decays[i] = slept /
508                         WORKFLOW_SCHEDULER_SLEEP_DECAY_TICKS;
509                     if (planned_decays[i] >
510                         WORKFLOW_SCHEDULER_MAX_SLEEP_DECAYS)
511                         planned_decays[i] =
512                             WORKFLOW_SCHEDULER_MAX_SLEEP_DECAYS;
513                     planned_vruntime[i] =
514                         workflow_scheduler_decay_toward(
515                             planned_vruntime[i],
516                             planned_vtime,
517                             planned_decays[i]);
518                 }
519             }
520         }
521         /* Urgency may shorten an open request, never mint more service. */
522         if (planned_remaining[i] == 0 ||
523             request_cycles < planned_remaining[i]) {
524             planned_remaining[i] = request_cycles;
525             planned_request_ticks[i] = workflow_scheduler_request_ticks(
526                 candidates[i].latency_class,
527                 candidates[i].deadline_delta_ticks);
528             planned_latency[i] = candidates[i].latency_class;
529         }
530         if (planned_vruntime[i] > ~0ULL - planned_remaining[i])
531             goto fail;
532         planned_deadline[i] =
533             planned_vruntime[i] + planned_remaining[i];

```

```

534     }
535     if (count == 1) {
536         if (planned_vruntime[0] > planned_vtime)
537             planned_vtime = planned_vruntime[0];
538     } else {
539         uint64 average = workflow_scheduler_average(
540             planned_vruntime, count);
541
542         if (average > planned_vtime)
543             planned_vtime = average;
544     }
545     for (uint i = 0; i < count; i++) {
546         if (planned_vruntime[i] <= planned_vtime) {
547             if (selected < 0 ||
548                 workflow_scheduler_before(candidates,
549                     planned_deadline,
550                     planned_vruntime, i,
551                     (uint)selected))
552                 selected = i;
553         }
554     }
555     if (selected < 0)
556         goto fail;

```

Code 20 : Workflow EEVDF 的提交、dispatch 与 charge

c

```

558     /* Phase two commits a plan that can no longer fail. */
559     if (rebase_offset != 0)
560         workflow_scheduler_rebase_locked();
561     for (uint i = 0; i < count; i++) {
562         struct workflow_scheduler_entity *entity =
563             &workflow_scheduler_entities[slots[i]];
564
565         if (new_slots & (1U << i)) {
566             memset(entity, 0, sizeof(*entity));
567             entity->lifecycle = candidates[i].lifecycle;
568             entity->account = candidates[i].account;
569             entity->domain_id = candidates[i].domain_id;
570             entity->flags = AGENT_WORKFLOW_SCHED_F_ACTIVE;
571             entity->earliest_deadline_tick =
572                 WORKFLOW_SCHEDULER_NO_DEADLINE;
573             entity->cached_latency_class =
574                 (uchar)candidates[i].latency_class;
575             entity->schedulable_threads =
576                 (ushort)candidates[i].runnable_threads;
577         }
578         workflow_scheduler_domain_slot[candidates[i].domain_id] =
579             (signed char)slots[i];
580         entity->vruntime = planned_vruntime[i];
581         entity->remaining_cycles = planned_remaining[i];
582         entity->virtual_deadline = planned_deadline[i];
583         entity->request_ticks = (uchar)planned_request_ticks[i];
584         entity->latency_class = (uchar)planned_latency[i];
585         entity->runnable_threads =
586             (ushort)candidates[i].runnable_threads;
587         entity->mode = AGENT_WORKFLOW_SCHED_MODE_EEVDF;

```

```

588     if (wake_transitions & (1U << i)) {
589         entity->sleep_decays = workflow_scheduler_counter32_add(
590             entity->sleep_decays, planned_decays[i]);
591         entity->wake_tick = now_tick;
592         entity->flags |= WORKFLOW_SCHEDULER_F_WAKE_PENDING;
593     }
594     entity->flags &= ~(AGENT_WORKFLOW_SCHED_F_SLEEPING |
595         AGENT_WORKFLOW_SCHED_F_ELIGIBLE |
596         AGENT_WORKFLOW_SCHED_F_FALLBACK);
597     entity->flags |= AGENT_WORKFLOW_SCHED_F_RUNNABLE;
598     if (planned_vruntime[i] <= planned_vtime)
599         entity->flags |= AGENT_WORKFLOW_SCHED_F_ELIGIBLE;
600     else
601         entity->eligibility_misses =
602             workflow_scheduler_counter32_add(
603                 entity->eligibility_misses, 1);
604     }
605     workflow_scheduler_vtime = planned_vtime;
606     if (candidates[selected].deadline_delta_ticks == 0) {
607         struct workflow_scheduler_entity *entity =
608             &workflow_scheduler_entities[slots[selected]];
609
610         if (!(entity->flags & WORKFLOW_SCHEDULER_F_DEADLINE_MISSED)) {
611             entity->flags |= WORKFLOW_SCHEDULER_F_DEADLINE_MISSED;
612             entity->deadline_misses = workflow_scheduler_counter32_add(
613                 entity->deadline_misses, 1);
614         }
615     }
616     *selected_domain = candidates[selected].domain_id;
617     intr_restore(enabled);
618     return 0;

```

Algorithm 4：资源与 CPU 的工作流级闭环

1. 合同将 node 的 exec/storage envelope 与 charge class 写入 claim。
2. Phase Lease 对本阶段的账户额度进行锁定、激活和结算；超限或状态失效则返回受控结果。
3. runnable thread 汇聚到 workflow entity，EEVDF 以 entity 的 lag/deadline 选择下一次服务。
4. 受合同执行路径的 Context 与 Evidence 记录 terminal 状态，使资源拒绝、取消和超时能回溯到具体 node/attempt。

3.8.7. Guest 运行时与工件发布

Nexus 用户态库会重新发现内核工具目录，逐项核对 ABI、id/name/parameter schema 的一致性；发起调用时又检查当前 Agent identity、产品角色和 required capability。双侧校验的作用不是替代内核，而是让模型适配层尽早排除不可能成功的工具调用。

Code 21 : Nexus 对内核工具目录的发现与一致性检查

c

```

681 int agent_nexus_tools_discover(void)
682 {
683     unsigned char seen[AGENT_TOOL_COUNT];
684     int count;
685
686     memset(nexus_tool_catalog, 0, sizeof(nexus_tool_catalog));
687     memset(seen, 0, sizeof(seen));
688     nexus_tool_catalog_count = 0;
689     count = tool_list(nexus_tool_catalog, AGENT_TOOL_COUNT);
690     if (count != AGENT_TOOL_COUNT)
691         return -1;
692     for (int i = 0; i < count; i++) {
693         const struct agent_tool_desc_v2 *descriptor =
694             &nexus_tool_catalog[i];
695         const struct agent_nexus_tool_spec *spec;
696
697         if (descriptor->version != AGENT_CALL_VERSION_V2 ||
698             nexus_tool_catalog[i].size != sizeof(nexus_tool_catalog[i]) ||
699             descriptor->tool_id <= 0 ||
700             descriptor->tool_id > AGENT_TOOL_COUNT ||
701             strlen(descriptor->name, sizeof(descriptor->name)) ==
702                 sizeof(descriptor->name) ||
703             strlen(descriptor->params, sizeof(descriptor->params)) ==
704                 sizeof(descriptor->params) ||
705             seen[descriptor->tool_id - 1] != 0)
706             goto invalid;
707         spec = &nexus_tool_specs[descriptor->tool_id - 1];
708         if (spec->tool_id != descriptor->tool_id ||
709             !nexus_text_equal_bounded(spec->name, descriptor->name,
710                                     AGENT_TOOL_NAME_SIZE) ||
711             !nexus_text_equal_bounded(spec->parameters,
712                                     descriptor->params,
713                                     AGENT_TOOL_PARAMS_SIZE))
714             goto invalid;
715         seen[descriptor->tool_id - 1] = 1;
716     }
717     nexus_tool_catalog_count = count;
718     return count;
719
720 invalid:
721     memset(nexus_tool_catalog, 0, sizeof(nexus_tool_catalog));
722     return -1;
723 }

```

Code 22 : Nexus typed tool call 的用户态封装

c

```

894 int agent_nexus_tool_call(const char *name, unsigned long long request_id,
895                          const struct agent_nexus_tool_argument *arguments,
896                          unsigned int argument_count,
897                          struct agent_response_v2 *response)
898 {
899     const struct agent_tool_desc_v2 *tool = agent_nexus_tool_find(name);
900     const struct agent_nexus_tool_spec *spec = agent_nexus_tool_spec_find(name);

```

```

901     struct agent_request_v2 request;
902     struct agent_nexus_artifact_actor identity;
903     struct agent_info info;
904     unsigned int role_bit;
905
906     if (tool == 0 || spec == 0 || response == 0 || request_id == 0 ||
907         argument_count > AGENT_TOOL_PARAM_MAX ||
908         (argument_count != 0 && arguments == 0) ||
909         (tool->flags & AGENT_TOOL_F_CALLABLE) == 0 ||
910         argument_count > tool->param_count ||
911         !nexus_schema_arguments_valid(spec, arguments, argument_count))
912         return -1;
913     if (agent_nexus_identity_current(&identity) < 0)
914         return -1;
915     role_bit = AGENT_NEXUS_TOOL_ROLE(identity.product_role);
916     memset(&info, 0, sizeof(info));
917     if (agent_info(&info) != 0 || !info.is_agent ||
918         info.agent_id != (int)identity.agent_id ||
919         (spec->product_role_mask & role_bit) == 0 ||
920         (info.capability_mask & spec->required_capabilities) !=
921         spec->required_capabilities)
922         return -1;
923     memset(&request, 0, sizeof(request));
924     memset(response, 0, sizeof(*response));
925     memset(nexus_tool_params, 0, sizeof(nexus_tool_params));
926     request.version = AGENT_CALL_VERSION_V2;
927     request.size = sizeof(request);
928     request.tool_id = tool->tool_id;
929     request.param_count = argument_count;
930     request.request_id = request_id;
931     request.params = argument_count ?
932         (unsigned long long)nexus_tool_params : 0;
933     strcpy(request.tool_name, tool->name);
934     for (unsigned int i = 0; i < argument_count; i++) {
935         struct agent_param_v2 *param = &nexus_tool_params[i];
936
937         param->version = AGENT_PARAM_VERSION;
938         param->size = sizeof(*param);
939         param->type = arguments[i].type;
940         strcpy(param->key, arguments[i].key);
941         if (arguments[i].type == AGENT_PARAM_UINT64) {
942             param->value_size = sizeof(unsigned long long);
943             param->value.uint64_value = arguments[i].number;
944         } else {
945             unsigned int length = strlen(arguments[i].text,
946                 sizeof(arguments[i].text)) + 1U;
947
948             param->value_size = length;
949             strcpy(param->value.string_value, arguments[i].text);
950         }
951     }
952     if (tool_call(&request, response) != 0 ||
953         response->version != AGENT_CALL_VERSION_V2 ||
954         response->size != sizeof(*response) ||
955         response->request_id != request_id || response->tool_id != tool->tool_id ||
956         strlen(response->tool_name, sizeof(response->tool_name)) ==
957         sizeof(response->tool_name) ||

```

```

958     !nexus_text_equal_bounded(response->tool_name, tool->name,
959                               AGENT_TOOL_NAME_SIZE))
960     return -1;
961     return 0;

```

工件发布使用 workflow-scoped edit lease 串行化发布者，并先写入无效 magic 的 pending header、payload 与 fsync，最后才写入有效 header。由于 uCore 没有原子 link/rename commit，该 ABI 不是原子发布：崩溃恢复时会留下不可读的 publish-once tombstone，而不是可读的半成品。这是当前实现的直接技术限制，而不是成功路径的隐含保证。

edit lease 的 commit/abort 允许当前 owner，或与 lease 同 scope、同 lifecycle 且持有 ORCHESTRATE 的 Agent；到期清理后查不到该 lease 时返回 NOT_FOUND，incarnation 或 version 不匹配返回 STALE。只有具备编排权限的 ORCHESTRATOR_BREAK 会释放活动 lease；声明的 BREAK_EXPIRED 与未知 flags 在当前实现中均不改变处理。

Code 23 : Nexus 工件的顺序写入与非原子发布边界

c

```

1518 static int nexus_artifact_store(
1519     struct agent_nexus_artifact_header *stored, const void *payload,
1520     unsigned int payload_size)
1521 {
1522     struct agent_nexus_artifact_header digest_header;
1523     struct agent_nexus_artifact_header pending_header;
1524     struct agent_file_edit_state store_lock;
1525     char path[AGENT_NEXUS_ARTIFACT_PATH_SIZE];
1526     unsigned int invalid_magic = 0;
1527     int lock_active = 0;
1528     int created = 0;
1529     int existing;
1530     int fd = -1;
1531
1532     if (payload == 0 || payload_size == 0 ||
1533         payload_size > AGENT_NEXUS_ARTIFACT_MAX ||
1534         agent_nexus_artifact_path(stored->handle, path) < 0)
1535         return -1;
1536     agent_nexus_sha256(payload, payload_size, stored->payload_sha256);
1537     memset(stored->manifest_sha256, 0, sizeof(stored->manifest_sha256));
1538     digest_header = *stored;
1539     agent_nexus_sha256(&digest_header, sizeof(digest_header),
1540                      stored->manifest_sha256);
1541
1542     /*
1543      * uCore has no atomic link/rename commit. A workflow-scoped edit lease
1544      * serializes every publisher. The final header is written last, so a
1545      * crash leaves an invalid, publish-once tombstone rather than a readable
1546      * partial artifact. AGENT_NEXUS_ARTIFACT_PUBLISH_IS_ATOMIC documents
1547      * this deliberately non-atomic MVP boundary.
1548      */
1549     fd = open(NEXUS_ARTIFACT_STORE_LOCK, O_CREATE | O_WRONLY);
1550     if (fd < 0)
1551         return -1;
1552     existing = close(fd);
1553     fd = -1;
1554     if (existing < 0)
1555         return -1;

```

```

1556     memset(&store_lock, 0, sizeof(store_lock));
1557     if (agent_file_edit_begin(NEXUS_ARTIFACT_STORE_LOCK, 0,
1558         AGENT_FILE_EDIT_MAX_TTL, &store_lock) !=
1559         AGENT_STATUS_OK)
1560         return -1;
1561     lock_active = 1;
1562     existing = open(path, O_RDONLY);
1563     if (existing >= 0) {
1564         (void)close(existing);
1565         goto fail;
1566     }
1567     fd = open(path, O_CREATE | O_WRONLY | O_TRUNC);
1568     if (fd < 0)
1569         goto fail;
1570     created = 1;
1571     pending_header = *stored;
1572     pending_header.magic = 0;
1573     if (nexus_write_all(fd, &pending_header, sizeof(pending_header)) < 0 ||
1574         nexus_write_all(fd, payload, payload_size) < 0 || fsync(fd) < 0)
1575         goto fail;
1576     existing = close(fd);
1577     fd = -1;
1578     if (existing < 0)
1579         goto fail;
1580     fd = open(path, O_WRONLY);
1581     if (fd < 0 || nexus_write_all(fd, stored, sizeof(*stored)) < 0 ||
1582         fsync(fd) < 0)
1583         goto fail;
1584     existing = close(fd);
1585     fd = -1;
1586     if (existing < 0)
1587         goto fail;
1588     if (agent_file_edit_commit(store_lock.lease_id,
1589         store_lock.base_version, &store_lock) !=
1590         AGENT_STATUS_OK)
1591         goto fail;
1592     lock_active = 0;
1593     return 0;
1594
1595 fail:
1596     if (fd >= 0)
1597         (void)close(fd);
1598     if (created && lock_active) {
1599         fd = open(path, O_WRONLY);
1600         if (fd >= 0) {
1601             (void)nexus_write_all(fd, &invalid_magic,
1602                 sizeof(invalid_magic));
1603             (void)fsync(fd);
1604             (void)close(fd);
1605         }
1606     }
1607     if (lock_active)
1608         (void)agent_file_edit_abort(store_lock.lease_id);
1609     return -1;
1610 }

```

4. 项目测试

测试从结构合同、交叉编译、RISC-V Guest 场景、Host 运行时和一次性性能实验五个层次展开。功能结果以真实的系统调用输出和协议 digest 为依据；性能数据保留逐样本来源，从而使配对比较、分布图和统计表能够回溯到对应 boot。

4.1. 功能与性能测试

4.1.1. 测试组织

AgentOS-uCore 的测试按“结构合同、交叉编译、RV64 Guest 场景、Host 运行时、综合性能”五个层次组织。结构合同检查 UAPI 尺寸、偏移与模块依赖；交叉编译将真实 kernel/user object 链接到 RISC-V 镜像；Guest 场景通过系统调用观察 identity、Context、VFS、IPC、resource 与 scheduler 状态；Host 测试覆盖 frame、provider、N1、artifact 和 validator。

模块	主要程序	关键场景
身份与 Context	agenttrust_ucore, agentscope_ucore, agentfinal_ucore	create/fork/exec/exit、generation reuse、Context snapshot/branch/eviction
结构化工具	agenttoolabi_ucore, agentcontract_ucore, agenttask_ucore	typed schema、V3 DAG、retry/cancel、SQ/CQ backpressure/resync
Provenance 与 Fence	agentfinal_ucore, fence/provenance fixtures	来源传播、效果前 gate、critical ring、challenge receipt
Live-Query FS	agentfs_ucore, agentbench_ucore	incarnation、scan/index 一致性、ENTER/UPDATE/LEAVE、resync
Loop 与 EEVDF	agentloop_ucore, agentsched_ucore, agent_eevdf_ucore	atomic wait/wake、timeout、heartbeat、thread amplification、Jain fairness
Console 与 Nexus	labdemo_ucore, agentnexus_ucore	多模块恢复、3 turn/4 role、replan、approval deny、session close

表 4.1: 六项核心机制的测试场景

当前版本通过 545 项 Agent 模块与 ABI 合同检查、114 项 Nexus 合同检查以及 93 个 Host 测试程序。strict Nexus replay 在 fresh QEMU boot 中完成 11 个 request digest、3 个 turn、4 个业务角色、失败后重新规划、一次审批拒绝与完整关闭。

4.1.2. 一次性增强实验

为了保留可用于高级图表的逐样本数据，我们在提交 2b14fb1f74b9bd093e6de939a16554620835699e 上完成一次性增强实验。实验使用 30 个 fresh QEMU boot，保存 33 份原始串口输出、19 个 CSV 数据表和 7,498 条结构化记录。每条记录都包含来源文件、来源 SHA-256 与原文序号。

同一 boot 中的 inner pair 用于观察稳定工作量下的重复波动，跨 boot 汇总时使用 source

实验族	设计	独立单位	样本量
Traversal/Indexed	16 个平衡 AB/BA paired boot	boot	16 个 core 与 E2E 配对
File catalog grid	3 × 4 参数网格，每格 15 个 inner pair	source boot	180 个查询配对
Batch/Scalar/SQ-CQ	4 boot，8 组旋转顺序	boot/block	96 条 16-op sequence
Workflow EEVDF	6 boot，并发度 1–4	boot/window 与 probe	504 条 exact wake probe

表 4.2: 一次性实验组成

boot 作为独立单位。所有图表直接从 CSV 数据表生成，使原始顺序、target、workload parameter 和 warning 字段保留在数据中。

4.1.3. Task 传输延迟

Task 实验使 compact batch、scalar V3 和 SQ/CQ 分别完成 16 次等价 ECHO，并使用旋转顺序平衡三条路径的先后次序。每条 sequence 记录完整 wall-time，同时验证 tool/status、Context sequence、Evidence ticket 和 result fingerprint 一致。

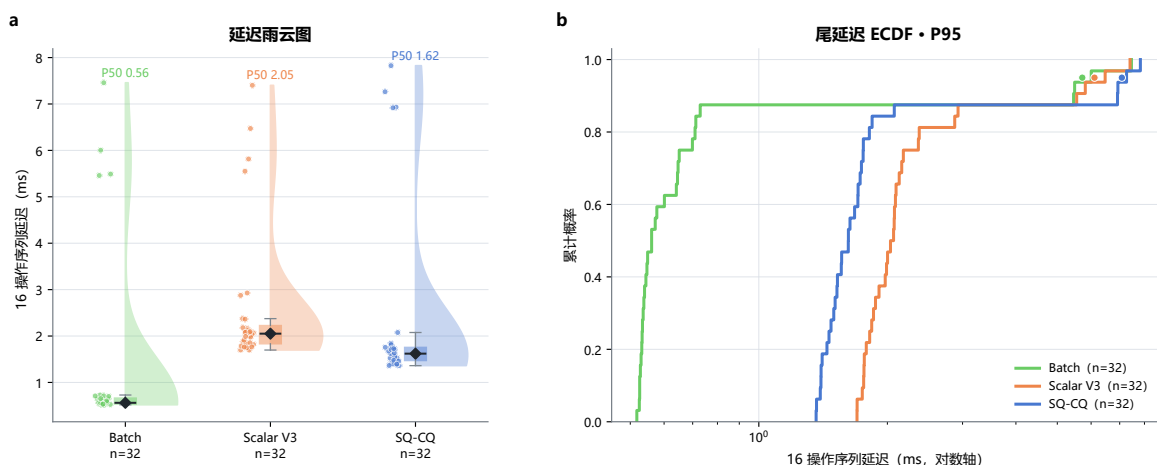


图 4.1: Task 延迟分布

Compact batch、scalar V3 和 SQ/CQ 的 sequence 中位数分别为 561 us、2051 us 和 1620.5 us。Compact batch 在一次 syscall 内顺序执行 16 项操作，入口开销最低；V3 每次绑定 contract/node/attempt 并执行完整合同检查；SQ/CQ 增加 ring 提交、completion 与 backpressure 语义，其延迟位于两条同步路径之间。

4.1.4. 内核 I/O 与工作量

性能实验同时记录文件读写、syscall、records examined、Task completion 与 workflow service 等计数。图4.2分为上下两个面板：上半图把 workflow-core 的 bytes_read 和 end-to-end 的全局 I/O 计数分别除以 core-window syscall；下半图按 completed operation 归一化 Agent Task 传输成本。

每一行使用独立量纲与色标，用于比较同一指标在不同路径下的相对变化。

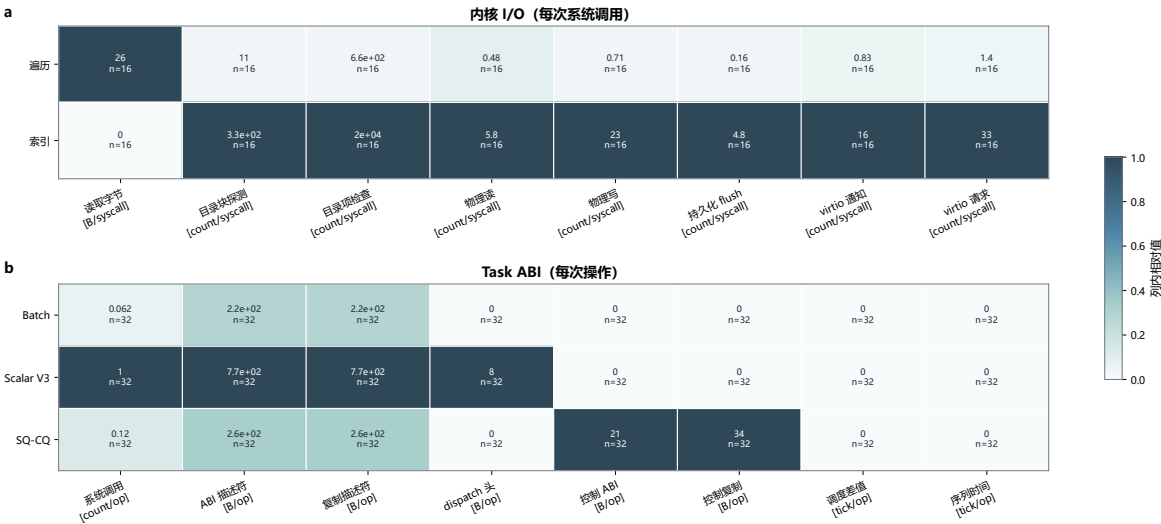


图 4.2: 内核 I/O 与工作量

4.1.5. 可重复性

一次性数据集的 manifest 记录源码 commit、工具版本、工作量计划、Guest 与构建哈希，并汇总原始数据、CSV 数据表和图像文件哈希。75 个 inventory 项目全部通过哈希校验。数据用于离线 validate 与 plot，日常回归继续运行常规 Guest 负载。

4.2. 测试路径与可复核证据

测试不只比对用户态输出文本，而是通过真实 RV64 Guest 系统调用观察内核对象的状态、返回码、Context sequence、generation 和运行统计。功能场景、机制探针和性能测量使用同一构建产物，保证表4.1中的项目能回溯到实际执行路径。

4.2.1. Native workload 与一次性测量

labdemo_ucore 的 native workload 在 Guest 内创建和使用 AgentOS 对象，收集机制路径上的计数与延迟。它与兼容 workload 共用报告结构，因此对比不会把 Host 端预处理、日志格式或不同的测试框架误当成内核性能差异。

Code 24 : Guest 内 Native AgentOS workload 的执行入口c

```
745 static void run_native_workload(struct labdemo_workload_metrics *metrics)
746 {
747     static struct agent_file_query query;
748     static struct agent_file_query_result result;
749     static struct agent_file_meta target;
750     struct labdemo_lane_measurement_state *state = &native_measurement;
751     static char name[6];
752
753     memset(metrics, 0, sizeof(*metrics));
754     demo_quiescence_fence("native", "E2E_START", 1, &state->e2e_start);
755     metrics->end_to_end_started_us = demo_now_us();
756     seed_native_workload(&target);
757     memset(&query, 0, sizeof(query));
```

```

758 query.flags = AGENT_FILE_QUERY_USE_INDEX;
759 query.max_hits = AGENT_FILE_QUERY_MAX_HITS;
760 strcpy(query.project, DEMO_PROJECT);
761 strcpy(query.run_id, DEMO_BENCH_RUN);
762 strcpy(query.status, "failed");
763 demo_quiescence_fence("native", "CORE_START", 2, &state->core_start);
764 metrics->started_us = demo_now_us();
765 DEMO_DIAG_BEGIN();
766 check(agent_file_query(&query, &result) == 1,
767       "native failed-stage query");
768 DEMO_DIAG_END("native", "failed_index_query");
769 metrics->records_examined += result.scanned_records;
770 check(result.returned == 1 && result.used_index == 1 &&
771       strcmp(result.hits[0].stage, "align") == 0 &&
772       strcmp(result.hits[0].status, "failed") == 0,
773       "native failed-stage result");
774 metrics->discovered_us = demo_now_us();
775 demo_corpus_name(name, 'n', DEMO_BENCH_TARGET);
776 DEMO_DIAG_BEGIN();
777 {
778     int fd = open(name, O_WRONLY | O_TRUNC);
779
780     check(fd >= 0, "native recovery open");
781     check(write(fd, DEMO_BENCH_RECOVERED_BODY,
782               strlen(DEMO_BENCH_RECOVERED_BODY)) ==
783           (ssize_t)strlen(DEMO_BENCH_RECOVERED_BODY),
784           "native recovery write");
785     target.update_mask = AGENT_FILE_META_UPDATE_STATUS |
786                       AGENT_FILE_META_UPDATE_SUMMARY;
787     strcpy(target.status, "recovered");
788     strcpy(target.summary, "memory_limit recovered");
789     check(agent_file_meta_set(&target) == 0,
790           "native recovery metadata stage");
791     check(demo_quiescence_fsync(fd) == 0,
792           "native recovery primary ack");
793     check(close(fd) == 0, "native recovery close");
794 }
795 DEMO_DIAG_END("native", "grouped_recovery_primary_ack");
796 metrics->committed_us = demo_now_us();
797 strcpy(query.status, "recovered");
798 memset(&result, 0, sizeof(result));
799 DEMO_DIAG_BEGIN();
800 check(agent_file_query(&query, &result) == 1,
801       "native recovered-stage query");
802 DEMO_DIAG_END("native", "recovered_index_query");
803 metrics->records_examined += result.scanned_records;
804 check(result.returned == 1 && result.used_index == 1 &&
805       strcmp(result.hits[0].stage, "align") == 0 &&
806       strcmp(result.hits[0].status, "recovered") == 0,
807       "native recovered-stage result");
808 metrics->result_items = 1;
809 metrics->outcome_hash = demo_outcome_hash(result.hits[0].stage,
810                                           result.hits[0].status);
811 metrics->finished_us = demo_now_us();
812 take_performance_snapshot(&state->core_ack);
813 metrics->workload_syscalls = performance_delta(
814     state->core_start.performance.snapshot.observer_workload_syscalls,

```

```

815     state->core_ack.snapshot.observer_workload_syscalls);
816     check(metrics->workload_syscalls != 0, "native workload syscall receipt");
817     demo_quiescence_fence("native", "ACK_SETTLED", 3,
818         &state->ack_settled);
819     DEMO_DIAG_BEGIN();
820     for (int i = 0; i < LABDEMO_CORPUS_SIZE; i++) {
821         demo_corpus_name(name, 'n', i);
822         check(unlink(name) == 0, "native corpus cleanup");
823     }
824     DEMO_DIAG_END("native", "cleanup_unlink");
825     demo_quiescence_fence("native", "E2E_END", 4, &state->e2e_end);
826     metrics->end_to_end_finished_us = demo_now_us();
827     print_workload_lane("native", metrics);
828     print_mechanism_delta("native", "core", &state->core_start.performance,
829         &state->core_ack);
830     print_mechanism_delta("native", "end_to_end",
831         &state->e2e_start.performance,
832         &state->e2e_end.performance);
833 }
834
835 static void make_op(struct agent_op *op, int tool, uint64 id, uint64 arg0,
836     const char *payload)
837 {
838     memset(op, 0, sizeof(*op));
839     op->version = AGENT_CALL_VERSION;
840     op->tool_id = tool;
841     op->request_id = id;
842     op->arg0 = arg0;
843     if (payload)
844         strcpy(op->payload, payload);
845 }

```

一次性实验图表由当前测量目录的原始数据生成；图4.3汇总了机制路径的时间与资源观察值。每项结果均应理解为该 QEMU/uCore 配置、该组 workload 和该轮运行的观测值，而非跨硬件平台的通用吞吐承诺。

4.2.2. 工具、合同与任务通道测试

Nexus 运行时在发起工具调用时将任务文本编码成固定结构，并通过 send_message 进入 typed tool 路径。由于这里先完成任务结构校验、参数编码和回包 status 检查，测试可以明确区分用户态编码失败、内核 schema 拒绝和工具效果失败。

Code 25 : Nexus 任务编码与 send_message 调用

c

```

992 int agent_nexus_task_send(int target_pid, unsigned long long task_id,
993     const struct agent_nexus_task *task,
994     struct agent_response_v2 *response)
995 {
996     struct agent_nexus_tool_argument arguments[2];
997     char message[AGENT_EVENT_PAYLOAD_SIZE];
998
999     if (target_pid <= 0 || task_id == 0 || response == 0 ||
1000         agent_nexus_task_encode(task, message) < 0)
1001         return -1;

```

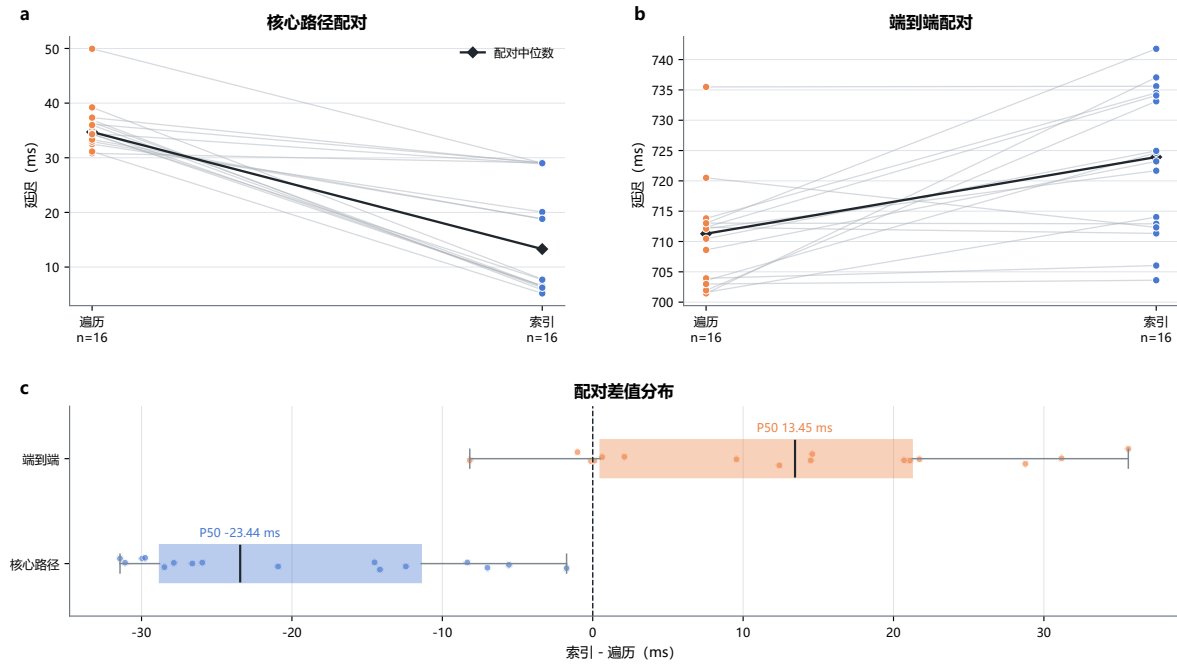


图 4.3: 当前构建下的 AgentOS-uCore 性能测量汇总

```

1002     memset(arguments, 0, sizeof(arguments));
1003     strcpy(arguments[0].key, "target_pid");
1004     arguments[0].type = AGENT_PARAM_UINT64;
1005     arguments[0].number = (unsigned long long)target_pid;
1006     strcpy(arguments[1].key, "message");
1007     arguments[1].type = AGENT_PARAM_STRING;
1008     strcpy(arguments[1].text, message);
1009     if (agent_nexus_tool_call("send_message", task_id, arguments, 2,
1010                             response) < 0)
1011         return -1;
1012     return response->status;
1013 }

```

合同测试覆盖 stale lifecycle、stale contract、未知 node、工具不匹配、deadline 和 dependency terminal 等分支。效果 gate 的终态会关联 evidence ticket 与 output provenance；对于 direct/V3 拒绝，critical record 无法建立时会转为 NO_SPACE/RETRY，测试也覆盖这条记录失败语义。因此拒绝或取消不是孤立的 errno，而是可与 Context/Evidence 或明确记录失败状态关联的结果。

Code 26 : 合同完成时的 Evidence 与 terminal 状态不变量

c

```

1834 void
1835 agent_execution_contract_complete(struct agent_execution_claim *claim,
1836                                 const struct agent_result *result,
1837                                 const struct agent_execution_outcome *outcome)
1838 {
1839     struct agent_execution_contract_record *record;
1840     struct agent_execution_node_internal *node;
1841     struct agent_execution_node_runtime *runtime;
1842     struct agent_execution_completion *cached;
1843     struct agent_execution_producer_metadata *producer;
1844     int cache_index;

```

```

1845     int terminal_failure;
1846     int enabled;
1847
1848     if (claim == 0 || result == 0 || !claim->active || claim->legacy ||
1849         claim->slot < 0 ||
1850         claim->slot >= WORKFLOW_LIFECYCLE_CAP ||
1851         claim->node_id >= AGENT_EXECUTION_CONTRACT_MAX_NODES)
1852         return;
1853     enabled = intr_save();
1854     record = &agent_execution_contracts[claim->slot];
1855     if (!agent_execution_record_matches(record, claim->lifecycle) ||
1856         record->generation != claim->contract_generation ||
1857         claim->node_id >= record->node_count) {
1858         intr_restore(enabled);
1859         return;
1860     }
1861     runtime = &record->runtime[claim->node_id];
1862     node = &record->nodes[claim->node_id];
1863     if (runtime->state != AGENT_EXECUTION_NODE_RUNNING ||
1864         runtime->attempt_id != claim->attempt_id ||
1865         !agent_execution_digest_equal(runtime->request_digest,
1866             claim->request_digest)) {
1867         intr_restore(enabled);
1868         return;
1869     }
1870     if ((runtime->flags &
1871         AGENT_EXECUTION_RUNTIME_F_CANCEL_REQUESTED) != 0 &&
1872         result->status != AGENT_STATUS_CANCELLED)
1873         panic("execution contract cancel winner");
1874     if ((runtime->flags &
1875         AGENT_EXECUTION_RUNTIME_F_TIMEOUT_REQUESTED) != 0 &&
1876         result->status != AGENT_STATUS_TIMEOUT)
1877         panic("execution contract timeout winner");
1878     if (result->sequence == 0 || outcome == 0 ||
1879         outcome->evidence_ticket == 0 ||
1880         outcome->output_provenance_labels == 0)
1881         panic("execution contract terminal without evidence");
1882     runtime->sequence = result->sequence;
1883     runtime->status = result->status;
1884     runtime->decision_reason = claim->decision_reason;
1885     record->running_mask &= ~(1ULL << claim->node_id);
1886     if (record->running_count == 0)
1887         panic("execution contract running underflow");
1888     record->running_count--;
1889     producer = &agent_execution_producers[claim->slot][claim->node_id];
1890     if (claim->producer_control_id == 0 || claim->producer_pid <= 0)
1891         panic("execution contract producer identity");
1892     producer->control_id = claim->producer_control_id;
1893     producer->pid = claim->producer_pid;
1894     producer->context_sequence = result->sequence;
1895     producer->provenance_labels = outcome->output_provenance_labels;
1896     producer->valid = 1;
1897     if (result->status == AGENT_STATUS_OK) {
1898         runtime->state = AGENT_EXECUTION_NODE_SUCCEEDED;
1899         record->completed_mask |= 1ULL << claim->node_id;
1900         record->failed_mask &= ~(1ULL << claim->node_id);
1901     } else {

```

```

1902     runtime->state = result->status == AGENT_STATUS_CANCELLED ?
1903         AGENT_EXECUTION_NODE_CANCELLED :
1904         AGENT_EXECUTION_NODE_FAILED;
1905     record->completed_mask &= ~(1ULL << claim->node_id);
1906     terminal_failure = claim->retry_forbidden ||
1907         claim->attempt_id >= node->max_attempts ||
1908         !agent_execution_retry_allowed(node, result->status);
1909     if (terminal_failure) {
1910         record->failed_mask |= 1ULL << claim->node_id;

```

Task Channel 在 submit 后维护 accepted/running 状态、生命周期引用与 callback 引用；hard deadline 触发 expire，complete/denied 进入统一 completion。对该状态机的测试同时验证“不丢生命周期引用”和“不在过期后伪装完成”两项性质。

Code 27 : Task submit 后的 callback、deadline 与完成分支

c

```

1880     request->sqe = sqe;
1881     request->accepted_tick = agent_task_now();
1882     request->expected_output_type = validation.output_artifact_type;
1883     request->provenance_labels = validation.output_provenance_labels;
1884     request->state = AGENT_TASK_REQUEST_ACCEPTED;
1885     request->flags = AGENT_TASK_REQUEST_F_LIFECYCLE_HELD;
1886     if (private->header.callback_refs == 0)
1887         panic("Task Channel submit transfer");
1888     agent_task_callback_put_locked(state);
1889     if (agent_task_request_input_acquire_locked(
1890         private, state->resource_private, request) < 0) {
1891         request->flags &= ~AGENT_TASK_REQUEST_F_LIFECYCLE_HELD;
1892         workflow_lifecycle_operation_leave(lifecycle);
1893         memset(request, 0, sizeof(*request));
1894         status = agent_task_channel_protocol_fault_locked(state);
1895         intr_restore(enabled);
1896         return status;
1897     }
1898     request->state = AGENT_TASK_REQUEST_RUNNING;
1899     private->header.sq_head++;
1900     private->header.submitted++;
1901     private->header.last_accepted_request_id = sqe.request_id;
1902     *consumed = 1;
1903     private->header.live_requests++;
1904     agent_task_callback_get_locked(state);
1905     agent_task_channel_publish_locked(state);
1906     intr_restore(enabled);
1907     hook_status = ops->submit(
1908         p, &sqe, &input_view, &validation, &completion);
1909     enabled = intr_save();
1910     state = agent_task_channel_find_locked(p);
1911     if (!agent_task_channel_owner_valid_locked(state, p) ||
1912         state->ops != ops || state->private->header.callback_refs == 0)
1913         panic("Task Channel submit callback");
1914     agent_task_callback_put_locked(state);
1915     intr_restore(enabled);
1916     agent_task_reclaim_deferred(p, ops);
1917     if (hook_status == AGENT_TASK_HOOK_PENDING &&
1918         (sqe.flags & AGENT_TASK_SQE_F_HARD_DEADLINE) != 0 &&

```

```

1919     agent_task_now() >= sqe.deadline_tick) {
1920         status = agent_task_channel_expire(p, agent_task_now(), ops);
1921         return agent_task_consumed_after_expire(status);
1922     }
1923     if (hook_status == AGENT_TASK_HOOK_COMPLETE ||
1924         hook_status == AGENT_TASK_HOOK_DENIED) {
1925         status = agent_task_channel_complete(
1926             p, sqe.ring_generation, sqe.request_id,
1927             sqe.slot_generation, &completion, ops);
1928         if (status == AGENT_TASK_CHANNEL_OK)
1929             return 1;
1930         return status;
1931     }
1932     if (hook_status != AGENT_TASK_HOOK_PENDING)
1933         return AGENT_TASK_CHANNEL_EVIDENCE;
1934     return 1;

```

4.2.3. Nexus 会话与 Host relay

Nexus 以带 sequence 的帧将 Guest 的会话、工具事件、审批和 telemetry 接入 Host relay。用户态 decoder 对 session、sequence、kind 与长度进行边界校验，再由 relay 进行 provider 适配和本地端点转发。这使网络/模型协议留在 Host，而 Agent identity、工具权限和任务状态仍在 Guest/Kernel 侧验证。

Code 28 : Nexus V2 帧的编码入口

c

```

903 static int live_frame_encode(const char *session, uint64 sequence,
904                             const char *kind, const char *payload,
905                             char *output, uint capacity)
906 {
907     struct live_builder builder;
908     unsigned char digest[LIVE_SHA_SIZE];
909     char digest_hex[LIVE_SHA_HEX_SIZE + 1];
910     uint payload_length = strlen(payload);
911     uint encoded_length;
912
913     if (!live_session_valid(session) || sequence == 0 ||
914         !live_kind_valid_v2(kind) ||
915         payload_length == 0 ||
916         payload_length > LIVE_MAX_JSON ||
917         !live_utf8_valid((const unsigned char *)payload, payload_length))
918         return -1;
919     live_sha256(payload, payload_length, digest);
920     live_digest_hex(digest, digest_hex);
921     encoded_length = live_base64_encode(
922         (const unsigned char *)payload, payload_length,
923         live_base64_buffer, sizeof(live_base64_buffer));
924     if (encoded_length == 0)
925         return -1;
926     live_builder_init(&builder, output, capacity);
927     live_builder_text(&builder, LIVE_PREFIX_V2);
928     live_builder_text(&builder, session);
929     live_builder_char(&builder, ' ');
930     live_builder_u64(&builder, sequence);

```

```

931     live_builder_char(&builder, ' ');
932     live_builder_text(&builder, kind);
933     live_builder_char(&builder, ' ');
934     live_builder_u64(&builder, payload_length);
935     live_builder_char(&builder, ' ');
936     live_builder_text(&builder, digest_hex);
937     live_builder_char(&builder, ' ');
938     live_builder_text(&builder, live_base64_buffer);
939     live_builder_char(&builder, '\n');
940     return builder.ok && builder.length <= LIVE_MAX_FRAME ?
941         (int)builder.length : -1;
942 }

```

Code 29 : Host relay 的串口写入与本地消息分发

Python

```

2154     def _reader(self, stream, source: str) -> None:
2155         def run() -> None:
2156             while True:
2157                 try:
2158                     chunk = stream.read(4096)
2159                     except OSError:
2160                         chunk = b""
2161                     if not chunk:
2162                         self.events.put(("eof", source.encode("ascii")))
2163                         return
2164                     self.events.put((source, chunk))
2165
2166                 threading.Thread(target=run, name=f"agentos-{source}", daemon=True).start()
2167
2168     def _write_serial(self, line: bytes) -> None:
2169         relay._write_process_before_deadline(
2170             self.process,
2171             line,
2172             deadline_monotonic=time.monotonic() + SERIAL_WRITE_TIMEOUT_SECONDS,
2173         )
2174
2175     def _drain_local(self) -> None:
2176         # Bound each pass so local traffic cannot defer provider polling or
2177         # Guest serial processing. The role-aware queue also prevents a ping
2178         # flood from placing a controller command behind every observer item.
2179         for _ in range(MAX_LOCAL_DRAIN):
2180             try:
2181                 peer, message = self.endpoints.inbound.get_nowait()
2182             except queue.Empty:
2183                 return
2184             try:
2185                 # A peer may disconnect after its reader queued commands.
2186                 # Identity and liveness are checked at execution time so a
2187                 # replacement controller never inherits the old queue.
2188                 if not self.endpoints.is_current(peer):
2189                     continue
2190             try:
2191                 self._handle_local(peer, message)
2192             except relay.RelayError as error:
2193                 peer.send(
2194                     {

```

```

2195         "type": "error",
2196         "code": error.code,
2197         "message": error.public_message,
2198     }
2199 )
2200 finally:
2201     consumed = getattr(peer, "inbound_consumed", None)
2202     if consumed is not None:
2203         consumed()
2204
2205 def _handle_local(self, peer: _Peer, message: Mapping[str, object]) -> None:
2206     message_type = message.get("type")
2207     if peer.role == "observer":
2208         if message_type == "ping" and set(message) == {"type"}:
2209             peer.send({"type": "pong"})
2210             return
2211         raise relay.WireProtocolError("ROLE_DENIED", "observer is read-only")
2212     if message_type == "user_message":
2213         if set(message) != {"type", "content"}:
2214             raise relay.WireProtocolError("BAD_LOCAL_MESSAGE", "user message is malformed")
2215         self.session.submit_user(message.get("content"))
2216     elif message_type == "command":
2217         if set(message) != {"type", "command"}:
2218             raise relay.WireProtocolError("BAD_LOCAL_MESSAGE", "control message is
malformed")
2219         command = message.get("command")
2220         self.session.request_control(command)
2221     elif message_type == "approval":
2222         binding_fields = (
2223             NEXUS_APPROVAL_BINDING_FIELDS
2224             if self.guest_profile == "nexus"
2225             else APPROVAL_BINDING_FIELDS
2226         )
2227         expected = {"type", "decision", *binding_fields}
2228         if set(message) != expected:
2229             raise relay.WireProtocolError("BAD_LOCAL_MESSAGE", "approval message is
malformed")
2230         self.session.decide_approval(

```

测试运行中还关注关停、取消和过期的收敛。Host relay 可请求停止，Guest 将等待和工具事件转化为可观测的终态；内核则以 lifecycle generation 阻止回收后的延迟消息、Task completion 或 watcher 引用命中新的 workflows。

4.2.4. 测试结论的解释范围

本项目的测试证明当前源码与所列 Guest workload 在测试环境中的机制连通性和约束分支可达性。它不把 Context rollback 描述为外部副作用撤销，也不把 Task resource import 或非原子 artifact publish 当作已完成的通用事务能力。后续应在真实多核、长时间压测、故障注入和持久化恢复条件下继续评估这些边界。

5. 总结与展望

5.1. 阶段性结论

我们在 RISC-V uCore 中实现了一套系统原生 Agent 运行机制。AgentOS-uCore 以现有进程、页表、VFS、syscall 和 scheduler 为底座，将身份、工具、Context、文件查询、事件等待、资源与调度组织为统一 workflow 主体。六组机制使 Agent 在真实系统效果发生时具有明确的 identity、capability、scope、generation、provenance 和 resource account。

产品的关键进展可以归纳为三点。第一，Agent 已经从应用层标签变成内核可识别的运行主体，普通进程和 Agent 可在同一 uCore 中共存。第二，typed V2/V3、Live-Query、MESSAGE wait、Credit Domain 与 EEVDF 覆盖了 Agent Loop 中从调用、等待到资源服务的主要数据路径。第三，Console 与 Nexus 将内核机制连接到综合科研恢复场景和四角色长驻协作，使产品能够同时支持确定性流程与模型驱动交互。

实验结果展示了 AgentOS 各条路径的性能特征。indexed core path 在 16 个独立配对中全部取得更低延迟，配对加速比中位数为 3.118 倍；12 组 catalog 网格的中位加速比范围为 1.164–2.808 倍；504 条 EEVDF wake probe 全部在 0–1 tick 内得到服务。端到端窗口中 indexed 的中位差值为 +13.452 ms，表明后续优化重点应向恢复写入、fsync 和复核阶段延伸。

下一阶段将沿三条主线继续推进：为 Task Channel 增加业务 payload backend，使 SQ/CQ 承载 Nexus artifact 任务；为 artifact 与 Workflow Fence 增加可选的持久化层，支持跨 boot 恢复；将 workflow EEVDF 扩展到 SMP 和真实 RISC-V 硬件，分析多核实体迁移、唤醒与资源记账。这些工作将继续使用 identity、lifecycle 和 workflow resource domain 作为统一主体，使 AgentOS 保持清晰、紧凑的内核结构。

5.2. 当前技术限制

Context rollback 恢复的是后续决策所见的 Context 记录与路径状态，不会撤销已经完成的外部效果；需要幂等的 effect 仍由 V3 attempt cache 与具体工具协议处理。Task Channel 已实现控制面、队列、deadline 与 completion 语义，但当前版本拒绝 resource import。artifact publish ABI 仍不是跨对象的原子提交，后续需要将 artifact、metadata、fence 和持久化事务进一步整合。

5.3. 后续工作

下一阶段将为 Task Channel 增加业务 payload backend，使 SQ/CQ 能承载 Nexus artifact 任务；为 artifact 与 Workflow Fence 增加可选持久化层，支持跨 boot 恢复；将 workflow EEVDF 扩展至 SMP 与真实 RISC-V 硬件，分析实体迁移、唤醒和资源记账。上述演进仍以 identity、lifecycle 和 workflow resource domain 作为统一主体。

6. 参考说明

6.1. uCore 与系统基础

AgentOS-uCore 建立在 LearningOS/uCore 教学内核之上，继承其 RISC-V 启动、进程、虚拟内存、VFS、trap、系统调用与基础调度框架。项目在这些现有对象上增加 Agent 状态与 UAPI，不修改其作为底座的基本责任边界。相关基础资料见参考文献中的 uCore 课程材料。

6.2. 协议、工具与运行时

结构化工具和 Task Channel 使用定长结构、version/size、共享头文件以及 Guest/Kernel 布局检查。Nexus 的 Host relay 管理本地 socket、TLS、provider adapter 与 frame 转发；Guest 保留业务状态、artifact 和内核交互。系统中的外部依赖及许可信息均以仓库中的许可证和源码声明为准。

6.3. 测量资料

性能图来自 docs/figures/performance/ 的当前导出文件。一次性增强实验保留原始串口输出、CSV 和逐样本 provenance，用于复核配对延迟、扫描组、Task sequence 与 EEVDF wake probe。图表叙述仅引用这些保存的测量结果，不以模型推断替代原始记录。

6.4. 文献使用

系统设计参考操作系统、能力安全、结构化接口、事件驱动运行时和公平调度的公开资料。

7. 参考文献

- [1] LearningOS Community. ucore-tutorial-code-2025s. <https://github.com/LearningOS/uCore-Tutorial-Code-2025S>, 2025. AgentOS-uCore 的教学内核基础; 访问日期: 2026-08-11.
- [2] Linux Kernel Documentation. Cpu accounting controller. <https://docs.kernel.org/6.11/admin-guide/cgroup-v1/cpuacct.html>, 2024. 访问日期: 2026-08-11.
- [3] Linux Kernel Project. rstat.c. <https://github.com/torvalds/linux/blob/master/kernel/cgroup/rstat.c>, 2026. 访问日期: 2026-08-11.
- [4] Linux Kernel Documentation. Bpf ring buffer. <https://docs.kernel.org/6.6/bpf/ringbuf.html>, 2023. 访问日期: 2026-08-11.
- [5] Linux Kernel Documentation. Eevdf scheduler. <https://docs.kernel.org/scheduler/sched-eevdf.html>, 2026. 访问日期: 2026-08-11.
- [6] Jens Axboe. Efficient io with io_uring. https://kernel.dk/io_uring.pdf, 2019. 访问日期: 2026-08-11.
- [7] Haiku Project. Practical file system design with the be file system. <https://www.haiku-os.org/legacy-docs/practical-file-system-design.pdf>, 1999. 访问日期: 2026-08-11.
- [8] Anthropic. How tool use works. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>, 2026. 访问日期: 2026-08-11.
- [9] Anthropic. How claude code works: The agentic loop. <https://code.claude.com/docs/en/how-claude-code-works>, 2026. 访问日期: 2026-08-11.